

AICD

Artificial Intelligence Centered Development

A software development methodology in which artificial intelligence builds, tests, operates and documents the software, and humans specify, verify, decide and supervise. A guide for teams that intend to ship and maintain production systems without writing code by hand.

by Alim Sahane

Version 0.3, working draft. September 2026.

This document is a living base. It records what has been decided so far and is extended as the methodology is explored and tested on real projects.

COLOPHON

AICD: Artificial Intelligence Centered Development

Version	0.3, working draft
Date	September 2026
Author	Ngatcha Sahane Rachid Alim, publishing as Alim Sahane
Repository	https://github.com/rachidsahane/AICD-METHODOLOGY

LICENSE

[AICD](#) © 2026 by [Alim Sahane](#) is licensed under [CC BY 4.0](#)  

Photographs and the author's likeness are excluded from this license and remain all rights reserved.

The name AICD is subject to the trademark policy in TRADEMARK.md.

This PDF is built from the HTML source in the repository above. The source at its release tag is the authoritative version of the methodology.

Contents

PART I: UNDERSTANDING AICD

1. Why a new methodology	5
2. AICD in one page	6
3. Founding principles	7
4. What stays, what changes	8
5. Design philosophy: open to extension, closed to modification	10

PART II: THE ARCHITECTURE OF AICD

6. The five layers	12
7. The agent team and separation of duties	13
8. Memory architecture	16

PART III: HOW WORK FLOWS

9. The specification system	19
10. Feature flow: from idea to observed behavior	20
11. Ticket lifecycle and categories	21
12. Working validated tickets: fleet mode, pairing mode, escalation	23
13. Git and release workflow	25
14. Verification: tests written in language, built by AI, checked by humans	26
15. The continuous test environment	28
16. Operations and the observability loop	30
17. Autonomy tiers and the permission model	31

PART IV: PEOPLE AND ORGANIZATION

18. Human roles	33
19. Hiring and keeping judgment sharp	34
20. Stack selection policy	35

PART V: GOVERNANCE, ECONOMICS, RESILIENCE

21. Economics and metrics	36
---------------------------	----

22. Failure modes and safeguards	38
 PART VI: APPLYING AICD	
23. Starting a new product under AICD	40
24. Migrating an existing product into AICD	44
 PART VII: DEEP DIVES	
25. Memory service: implementation model	50
26. Operating several products with one team	52
27. Security and compliance in depth	52
28. The fleet dashboard	54
29. Evolving the methodology: the extension process	55
30. Calibration procedures	56
 PART VIII: EXTENSIONS AND PROFILES	
31. Simulation personas and finding quality	58
32. Agent instruction design	60
33. Design in the loop	61
34. The reference organizational repository	61
35. Profile: products with mobile clients	62
36. Profile: operating without live telemetry	65
37. Profile: regulated domains	65
38. Profile: the single operator	66
39. Lessons from the first application	67
40. Open questions and next explorations	68
 APPENDICES	
A. Templates	70
B. Glossary	72
C. Document history	74
About the author	76

PART I

Understanding AICD

This part is written for everyone: founders, managers, product people, investors and the engineers who will work inside the methodology. It explains why AICD exists, what it is, and what it asks of the humans who use it. No technical background is required.

1 Why a new methodology

Every software development methodology of the last forty years was designed around one scarcity: the time of skilled humans writing code. Waterfall planned that time in advance. Agile and Scrum sliced it into sprints and estimated it in story points. Daily standups, retrospectives, velocity charts and backlogs all exist to coordinate and protect a resource that was expensive, slow, and hard to hire.

That scarcity is disappearing. A modern AI coding agent, given a clear description of what should exist, produces working software in hours, in any well-documented programming language, at a quality level that most working developers do not reach. It writes the tests, reads the error logs, fixes what it broke, and documents what it did. The cost of producing code has collapsed.

When the scarce resource disappears, the methodology built around it stops making sense. Teams that keep Scrum and simply add an AI assistant get a faster version of the old bottleneck, not a new way of working. The real question is: once producing code is cheap, where does the difficulty go?

WHERE THE DIFFICULTY MOVES

Specification. Describing precisely what should exist, including edge cases, failure modes and security expectations. The AI builds exactly what is described; an imprecise description produces a precise implementation of the wrong thing.

Verification. Proving, with confidence, that what was built is correct, secure, performant, and does not break what already worked. When humans no longer read most of the code, verification must happen at the level of behavior and evidence.

Decision. Choosing the architecture, the technology stack, the infrastructure, and what is worth building next. These are judgments about trade-offs, risk and business value, and they remain human.

AICD is a methodology designed around these three new bottlenecks rather than the old one. It treats code as a manufactured output, the way a factory treats a finished part, and it puts the humans where their judgment matters: at the specification, at the control points, and at the decisions.

The situation that produced AICD

AICD was not designed in the abstract. It comes from the experience of building several production SaaS products end to end with AI coding agents, deploying them to real users, and then facing the question every founder in that position faces: how do you build a team to maintain and grow software that no human wrote? The traditional answer (hire developers, give them the repository, run sprints) fails immediately. The developers cannot work faster than the agent that built the product, and the moment they start hand-editing generated code the product becomes a hybrid that neither the humans nor the agents fully understand.

The answer AICD proposes is different: build a team of experienced engineers who never write code, and give them a complete AI development organization to direct. The engineers specify, verify, decide and supervise. The AI builds, tests, operates and documents. The methodology defines how these two sides work together safely, on real products, at production quality.

2 AICD in one page

Artificial Intelligence Centered Development is a methodology in which a complete software development team is reproduced with AI agents, each with its own role and its own context, and supervised by a small team of experienced human engineers who act as specifiers, verifiers, decision-makers and monitors.

What the AI does

- **Builds** the software from written specifications, on isolated branches, through pull requests.

- **Tests** it, from acceptance criteria written by humans in plain language, across every test type: unit, integration, end-to-end, security, load, chaos and simulation.
- **Operates** it, watching production monitoring continuously, classifying errors, opening tickets, and fixing what it is authorized to fix.
- **Documents** it, keeping the specification synchronized with the code after every change.
- **Reviews** its own work, with a lead agent that checks the coder agents against the specification and escalates architectural questions to humans.

What the humans do

- **Specify:** write the product intent, the architecture decisions and the acceptance criteria, in collaboration with the AI.
- **Verify:** check that the test plan covers everything the specification demands, and approve pull requests based on evidence, not on reading every line.
- **Decide:** approve tickets that change behavior, architecture, data, security or cost; choose the stack; arbitrate escalations.
- **Supervise:** watch the fleet of agents, the production systems and the costs, and intervene when something drifts.

What the humans never do

Humans do not write production code and do not write test code. This is not a preference; it is a structural rule. The moment a human edits generated code by hand, the specification stops being the source of truth, and the agents lose the ability to reason about the system. Humans remain developers in the full sense of the word: they read code fluently in any language, they understand system design deeply, and they are hired for exactly that. They simply do not type the implementation.

THE ONE-SENTENCE VERSION

AICD takes a complete development team, automates every producing role with AI, and repositions the humans as supervisors, controllers and co-architects of the system.

3 Founding principles

These principles are the fixed base of AICD. Everything else in this document is a consequence of them, and every future extension of the methodology must be compatible with them.

1

Humans never write production code or tests.

Humans specify, verify, decide and monitor. Producing code and tests is entirely delegated to AI agents. Human ability to read and judge code remains mandatory.

- 2 The specification is the source of truth.**
Code is a generated artifact of the specification. If a behavior is not in the specification, it should not be in the system. If the code and the specification disagree, the specification is corrected first, and then the code is regenerated to match.
- 3 Every change enters through git under a defined workflow.**
No exception, human or agent. Version control is the spine of the methodology: it is the audit trail, the control point, and the rollback mechanism.
- 4 The stack is chosen per project, on merit, and recorded.**
Technologies are selected from mature, documented, production-proven options because they deliver the best result for the project, never because they are popular. Every selection is recorded in an architecture decision record.
- 5 Nothing ships without human verification.**
A human who understands the system design approves every behavioral change, based on the test plan, the coverage evidence and the agent's report. Reading every line is not required; understanding what was proven is.
- 6 Production is observed continuously and feeds the specification.**
Errors, usage signals and performance data flow back into tickets, then into the specification and the acceptance criteria. Incidents become permanent coverage, not one-off patches.
- 7 Every producing agent is separated from the agent that judges it.**
The agent that writes code is never the agent that decides whether it is correct. Coders, reviewers, testers and operators run with separate roles, separate contexts and separate permissions.
- 8 The methodology is model-agnostic and grows with model intelligence.**
AICD depends on no specific AI vendor or model. More capable models must make the methodology easier to run, never break it. Structures that exist for control (tickets, tiers, reviews) remain regardless of how smart the agents become.

4 What stays, what changes

AICD does not discard software engineering. It keeps the disciplines and changes the artifacts, the speed and the division of labor. The table below maps the traditional practices onto their AICD counterpart.

Traditional practice	What it was for	In AICD
----------------------	-----------------	---------

Sprints, story points, velocity	Coordinating and estimating scarce human coding time	Removed. Work flows continuously, ticket by ticket. Cost is measured in tokens and human approval time, not in estimated effort.
Daily standup, retrospective	Synchronizing humans who cannot share a brain	Replaced by the fleet dashboard, the human review queue with a fixed cadence, and human-owned post-mortems.
Git, branching, pull requests	Coordinating changes and keeping history	Kept and strengthened. Becomes the central control point. Agents produce many more changes than humans did; the workflow is stricter, not looser.
UML, Merise, domain modeling	Thinking about structure and data before building	Kept as thinking tools. Entity models, state machines and sequence diagrams live in the specification in text or Mermaid form, written with the AI, and are consumed directly by the agents.
Software development life cycle	Ordering requirements, design, build, test, deploy, maintain	Kept, compressed and repeated per feature. The whole cycle runs in hours per ticket instead of months per release.
Code review	Catching defects and design errors	Split. A lead agent reviews line by line against the specification. Humans review behavior, architecture and evidence, and read code only where risk justifies it.
Manual and automated testing	Proving correctness	Fully automated and expanded to simulation, chaos and continuous security testing. Humans author acceptance criteria and verify the test plan, never the test code.
Architecture decision records	Recording why choices were made	Central. ADRs are how humans constrain the agents and how the lead agent knows when to escalate.
On-call, incident response	Keeping production healthy	Agent-first. The operations agent detects, triages and, within its authorization, repairs. Humans handle escalations and own post-mortems.

Documentation	Explaining the system to future maintainers	Becomes the system's source code. Maintained by a documentation agent on every merge and audited for drift weekly.
---------------	---	--

5 Design philosophy: open to extension, closed to modification

AICD is meant to be as stable as the methodologies it replaces. A team should be able to adopt it, run a serious project on it, and never need to reinvent it mid-project. To achieve that, the methodology borrows a principle from software design itself: it is open to extension and closed to modification.

Closed to modification

The founding principles in section 3, the five-layer architecture in section 6 and the separation of duties in section 7 are the fixed core. They are not revised when a new model is released, when a new tool appears, or when a project finds them inconvenient. A project that needs to violate them is a signal that the project is not yet ready for AICD, or that the violation should be proposed as a methodology extension, reviewed, and adopted deliberately.

Open to extension

Everything that depends on the current state of tooling is an extension point: which models run which agents, which observability platform is used, how the memory service is implemented, what the test environment runs on, how tickets are stored. These are documented as organizational decisions and can change without touching the core.

Growing with model intelligence

Every structure in AICD exists for one of two reasons: either to give humans control, or to compensate for a current limitation of the agents. The two must be kept distinct.

- **Control structures** (tickets, risk tiers, human approval, separation of duties, permission model, audit trail) exist so that humans remain accountable for what runs in production. They stay in place no matter how capable the models become. A smarter agent makes them cheaper to run, because fewer escalations are needed and reviews are faster, but it does not remove them.
- **Compensation structures** (attempt budgets, detailed test plans before build, drift audits, cross-model review) exist because today's agents sometimes loop, drift or share blind spots. As models improve, the parameters of these structures loosen (larger budgets, fewer audits), but the structures themselves stay available, because the cost of keeping them is small and the cost of an unnoticed regression is not.

The practical rule: when a new model arrives, the team tunes parameters. It never removes a control structure, and it removes a compensation structure only after evidence, on a real project, that the limitation it compensated for is gone.

PART II

The architecture of AICD

The structural core of the methodology: the layers the work is organized in, the agents that do the work and how they are separated, and the memory that gives every agent knowledge of the whole project.

6 The five layers

AICD organizes every project into five layers. Each layer has its own artifacts, its own tools and its own human role. A change to the system always travels through the layers in order: it is specified, then executed, then verified, then operated, and governance rules apply at every step.

Layer 1: Specification

Human-owned. The real source code of the project.

Living documents per product: product brief, architecture document, architecture decision records (ADRs), domain model, API contracts, conventions, runbooks, and the agent instruction file that tells every agent how this project works. Rule: if it is not in the specification, the agents do not build it. Code is treated as a build artifact of this layer.

Layer 2: Execution

Agent-owned. Where code is produced.

Coder agents working with their tool connections (repositories, CI, documentation, ticketing), each on an isolated branch created from a ticket. Every change lands through a pull request. Agents never touch the main branch directly. A lead agent orders the work, assigns it, and reviews the result.

Layer 3: Verification

Shared. Humans define what must be proven; agents prove it; humans confirm the proof is complete.

Acceptance criteria written by humans before build. Test plans, test code and continuous testing produced by agents. Security scanning, performance budgets, mutation testing. Human review focused on behavior, coverage and architecture rather than on lines of code.

Layer 4: Operations

Agent-first. Humans handle escalations.

Error tracking, product analytics, logs, metrics, uptime and cost monitoring. An operations agent detects incidents and repairs within its authorization; a QA agent detects defects; a product signal agent detects usage patterns. All findings become categorized tickets. The loop closes when the fix is followed by a specification update and new acceptance criteria.

Layer 5: Governance

Human-owned. The rules that make the other four layers safe.

Autonomy tiers, the permission model for agents, secrets management, the immutable audit trail of agent actions, cost tracking, and the stack selection policy. Governance is enforced by infrastructure, not by instructions in a prompt.

7 The agent team and separation of duties

AICD reproduces a complete development team with AI agents. The word "team" is deliberate: each agent has one role, its own instructions, its own context, and its own permissions. This is the single most important structural decision in the methodology, for the same reason it matters in human organizations: the person who does the work must not be the person who judges it.

RULE: SEPARATION OF DUTIES

The agent that writes code never decides whether that code is correct. The agent that finds a defect never fixes it. The agent that deploys never changes code. Each of these roles runs in a separate context and with a separate, minimal set of permissions.

The roles

Agent	Responsibility	Reads	Writes
Coder	Takes one validated ticket, produces a plan, implements code and tests on its own branch, opens a pull request with a full report. Several coders run in parallel.	Product specification, conventions, its ticket, operational memory for the modules it touches	Its own branch only. Never main, never production.
Lead / Reviewer	Orders the ticket queue to avoid conflicts, assigns tickets to coders, reviews every pull request against the ticket, the specification and the ADRs, gives feedback to coders, escalates to humans when a decision is needed.	Everything in the specification and operational memory for its product, plus organizational knowledge	Review comments, ticket state, tier 0 merges only
QA / Tester	Turns human acceptance criteria into test plans, runs the continuous test environment (end-to-end, security, load, chaos, simulation), files categorized tickets with reproduction steps and evidence.	Specification, acceptance criteria, production telemetry (read only), the full defect history	Staging environment, tickets. No code repositories.
Operations	Watches production health against service level objectives, detects incidents, deploys from tags, rolls back, executes runbooks, opens incident tickets.	Runbooks, infrastructure ADRs, incident history, all telemetry	Deployments, rollbacks, infrastructure within runbooks. No code changes.
Documentation	On every merged pull request, proposes the specification update (or confirms none is needed). Runs the weekly drift audit between specification and code.	Specification, merged diffs, code	Specification pull requests
Product signal	Watches usage analytics for patterns (drop-offs, unused features, unusual behavior) and files product signal tickets for the product owner. Never files tickets for coders.	Product analytics, product brief	Product signal tickets

Assistant	The human's thinking partner for exploring new features, evaluating stacks, drafting specifications and acceptance criteria. Has broad read access and no write access to anything except drafts.	Everything the human can read	Drafts only
------------------	---	-------------------------------	-------------

Review is three layers, not one

A second model reviewing the first is necessary and not sufficient: commercial models share training data, alignment methods and coding heuristics, so two of them can share a blind spot on concurrency, on a subtle data-loss path, or on an anti-pattern both learned as normal. AICD therefore requires that pull request review be three layers, each catching what the others cannot:

1. **Deterministic checks** that no model can talk its way past: static analysis, type checking, structural linting, a structural diff summary of what changed (interfaces, signatures, schema, dependencies, permissions), contract checks against the interface specifications, dependency and secret scanning, and mutation testing of the tests themselves.
2. **Adversarial model review** by the lead agent, on a different model than the coder, working through an explicit adversarial checklist rather than producing a summary: how could this lose data, how could this be exploited, what happens under concurrency, what happens on partial failure, what does this change that the ticket did not ask for, which test would have to be weakened for this to pass. The checklist is organizational knowledge and grows from post-mortems.
3. **Human review** according to tier, reading evidence for tier 1 and the diff itself for tier 2.

Why the lead agent runs on a different model

If the coder agents and the reviewing agent share the same underlying model, they share the same blind spots, and a review by the same intelligence that produced the code is weaker than it looks. AICD requires that the lead/reviewer run on a different model family than the coders it reviews. This is cheap, and it is the closest thing to a second opinion that an automated pipeline can have. Human review of tier 2 changes provides the third opinion.

What is deliberately not copied from human teams

Human teams have meetings, handoffs and rituals because humans cannot share a brain. Each separation in the agent team must justify itself as a control mechanism, not as a habit. Tickets stay because they are a human control point. Pull requests stay because they are the review and audit point. Separate contexts stay because of separation of duties. A "daily standup between agents" or a "sprint planning among agents" would be theater and is not part of AICD.

8 Memory architecture

The whole methodology assumes that the agents actually know the project: its intent, its architecture, its history, its conventions, and what has already been tried. This is the most underestimated part of running AI development at production quality. An agent with a shallow or stale view of the project produces confident, well-written code that quietly contradicts decisions made months earlier.

AICD therefore defines a unified global memory, structured in four layers and served to each agent through a scoped memory service. The distinction between the layers is who writes them and how they change.

The four layers of memory

Layer	Contents	Where it lives	How it changes
1. Canonical knowledge per product	Product brief, architecture, ADRs, domain model, API contracts, conventions, runbooks, agent instruction file	The product repository, versioned in git	Only through pull requests, under the same risk tiers as code. This is the source of truth.
2. Organizational knowledge whole company	The AICD methodology itself, cross-product conventions, stack ADRs, security policy, escalation rules, agent role definitions	One organizational repository, versioned in git	Through pull requests, approved by the architect and governance roles. Read by every agent in every product.
3. Operational memory per product	Tickets and outcomes, incidents and post-mortems, escalation decisions, approaches that were tried and failed, agent closing reports	Append-only structured log	Agents write freely as they work. Never edited, only appended. This is where "we tried that and it broke X" lives.
4. Working memory per agent, per task	The agent's scratchpad for the current ticket: plans, intermediate reasoning, partial results	Ephemeral	Discarded when the ticket closes, except what the agent promotes into its closing report in layer 3.

The memory service

On top of the four layers sits a memory service, exposed to the agents as a tool connection. It has three jobs:

1. **Index** layers 1 to 3: full-text search, semantic search, and a structural map of each code repository (modules, dependencies, entry points).
2. **Enforce scopes**: each agent role sees only what its role needs, as defined in the table in section 7. Scopes are enforced by the service, not by asking the agent to ignore what it should not see.
3. **Answer task-oriented questions**: "what do I need to know to work on ticket 482" returns the relevant specification sections, the relevant ADRs, the related past tickets and the code map of the affected modules, instead of handing over the entire project and hoping the agent finds the right parts.

The sanitization barrier between telemetry and memory

Production data is untrusted input (section 16). If raw error messages, stack traces, user content or session replays were appended to operational memory as they arrive, an injected string in a crash log would become part of the context served to coders and reviewers on unrelated tickets, indefinitely. AICD therefore places a deterministic barrier between layer 4 and layer 3:

- Only **typed, structured records** enter operational memory and its index: ticket, category, severity, the journey step, the error class, a normalized fingerprint, affected module, counts, timestamps. Fields are parsed into a fixed schema by deterministic code, never summarized into memory by a model.
- **Raw evidence** (full traces, payloads, replays, user content) is stored out of band in an evidence store, referenced from the record by identifier, never indexed for retrieval, and shown to a human or an agent only on explicit request for that ticket, quoted and labeled as untrusted.
- Free-text fields that must be kept (an error message string, for example) are length-capped, stripped of anything that is not part of the declared format, and stored with a provenance tag that the retrieval API surfaces to every consumer.
- The barrier is code, versioned in the organizational repository, with its own tests, including planted injection strings that must be rejected.

Keeping memory alive

Memory that is not maintained is worse than no memory, because agents trust it. Two mechanisms keep it honest:

- **Documentation agent on every merge**. Each merged pull request triggers a proposed update to canonical knowledge, or an explicit statement that no update is needed. The proposal is itself a pull request.
- **Weekly drift audit**. The documentation agent compares the canonical documents with the actual code and flags every divergence to a human. Every canonical document carries a "last verified against code" date.

WHY GIT-BACKED MARKDOWN AND NOT ONLY A DATABASE

Layers 1 and 2 are kept as versioned text in git on purpose. It gives history, review through pull requests, diffs a human can read, and independence from any single memory tool. The index in the memory service is derived from these files and can be rebuilt at any time. The database is a cache; the repository is the truth.

PART III

How work flows

The operational heart of AICD: how a need becomes a specification, a ticket, a branch, a verified pull request, a deployment and an observed behavior, and what the humans and the agents do at every step.

9 The specification system

Because the specification is the source of truth, it needs the discipline that source code has: a fixed structure, clear ownership, versioning and review. Every product has the same set of documents. Templates are organizational (layer 2 of memory); contents are per product (layer 1).

Document	Owner	Purpose
Product brief	Product owner	Who the product is for, the problem it solves, what is in and out of scope, business constraints. The boundary inside which behavioral tickets can be built without a decision.
Architecture document	Architect	System overview, components, data flows, deployment topology, update strategy, non-functional targets (performance, availability, security). Drafted with the AI, decided by the human.
Architecture decision records	Architect	One record per significant decision: context, options considered, decision, consequences. The lead agent treats every ADR-covered area as an escalation trigger.

Domain model	Architect (generated by AI, reviewed)	Entities, relationships, invariants, state machines. Written in text and Mermaid so it is both human-readable and agent-consumable.
API and interface contracts	Architect (generated by AI, reviewed)	Public and internal interfaces, event schemas, error semantics. Changing a contract is always a decisional ticket.
Acceptance criteria	Verification lead	Per feature and per ticket: observable behaviors, edge cases, failure modes, security expectations. The one artifact humans author fully (with AI assistance). Detailed in section 14.
Conventions and agent instructions	Architect	Coding conventions, project structure, how to run tests, what the agents must always and never do in this repository.
Runbooks	Reliability and governance	Operational procedures the operations agent is allowed to execute: deploy, rollback, scale, rotate, recover.

Rules of the specification system

- Specification changes are pull requests, reviewed and merged under the same risk tiers as code.
- Every document carries a "last verified against code" date maintained by the drift audit.
- A ticket that changes behavior must reference the specification section it implements or modifies. A ticket with no specification anchor is either a defect (the specification was violated) or a decisional ticket (the specification must change first).

10 Feature flow: from idea to observed behavior

AICD replaces sprints with a continuous flow. Each feature, fix or improvement travels the same path, and the whole software development life cycle runs inside it, in hours or days rather than months.



Step	What happens	Who
Specify	The need is written into the product brief, the architecture and the acceptance criteria. New features are explored with the assistant agent: options, stacks, infrastructure, trade-offs. The human decides.	Product owner, architect, verification lead, with the assistant agent

Plan	The ticket is created and categorized. The coder agent produces an implementation plan and the QA agent produces a test plan. Both are reviewed before any code exists.	Lead agent proposes, humans approve according to category
Build	The coder implements code and tests on its branch, commits with traceability, opens a pull request with a full report.	Coder agent
Verify	CI runs every gate. The lead agent reviews line by line. The human reviews the coverage matrix, the CI report and the agent report, and approves according to the risk tier.	Lead agent, then human according to tier
Ship	Merge to main, tag, deploy from the tag. Rollback is always "redploy the previous tag".	Operations agent
Observe	Error tracking, analytics, metrics and the continuous test environment watch the change in production and staging.	Operations, QA and product signal agents
Feed back	Findings become categorized tickets. Every fix ends with a specification update and new acceptance criteria that would have caught the issue.	Documentation agent, verification lead

11 Ticket lifecycle and categories

Tickets are the control point of AICD. Every unit of work, whether requested by a human or discovered by an agent, is a ticket. Because agents that run continuously produce far more findings than a human team ever would, tickets must be categorized so that human attention goes only where a decision is actually needed.

The four categories

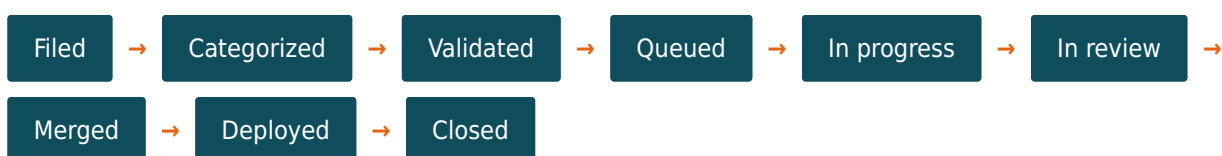
Category	Definition	Examples	Human involvement
----------	------------	----------	-------------------

Auto	A clear defect, reproducible, whose fix stays entirely within the current specification.	Crash, failing background job, flaky test, dependency patch, log noise	None before work. Goes straight to the coder queue. Reviewed by the lead agent; merged under tier 0 or 1 rules.
Behavioral	A fix or improvement that changes observable behavior but stays inside the product brief and the architecture.	Validation message change, new filter on an existing list, performance optimization that changes a response shape	Coder may build immediately. A human approves the pull request.
Decisional	Touches the specification, the architecture, the data model, security, external cost, a contract, or any tier 2 area.	New entity, new payment flow, schema migration, new external service, change to authentication	A human approves the ticket before any work starts. Usually handled in pairing mode.
Product signal	Not a defect: an insight about usage or value.	Users abandon at step 3 of onboarding; a feature is never used; a segment behaves unexpectedly	Goes to the product owner only. Never enters the coder queue directly.

RULE: UPGRADE ONLY

The QA agent proposes the category when it files a ticket. The lead agent may upgrade a category (Auto to Behavioral, Behavioral to Decisional) but never downgrade it. Only a human can downgrade. This guarantees that an agent can always make a ticket safer and can never make it riskier on its own.

Lifecycle



"Validated" means the ticket is authorized to be worked: automatic for Auto and Behavioral, human-approved for Decisional. "Closed" requires the documentation agent's specification update (or its explicit "no change needed") and, for defects, new acceptance criteria that cover the case. A ticket cannot be closed with a fix alone.

Mandatory ticket contents

- Category and proposed risk tier
- Specification anchor: the section implemented, violated or to be modified
- For defects: reproduction steps, evidence (logs, traces, screenshots, session replays), severity, affected users if known
- For features: link to acceptance criteria
- Budget: maximum attempts, wall-clock time and tokens before the coder must stop and report

12 Working validated tickets: fleet mode, pairing mode, escalation

Once a ticket is validated, there are two ways to work it. They are two modes of the same pipeline, not competing options.

Fleet mode (default)

Validated tickets enter a queue. The lead agent orders the queue to minimize conflicts, since two coders modifying the same module at the same time is the main practical problem of parallel agents. It assigns each ticket to a coder agent, which works on its own branch. When the coder opens a pull request, the lead agent reviews it against the ticket, the specification and the ADRs, sends feedback or questions back to the coder, and, once satisfied, either merges (tier 0) or hands to a human (tier 1 and 2). Humans see the fleet through a dashboard: queue depth, tickets in progress, blocked items, pull requests awaiting approval, cost.

Conflict handling at scale

Ordering the queue by hand is enough for a handful of coders and becomes a bottleneck beyond that. AICD does not rely on serialization; it relies on declared scope. Every implementation plan declares the modules it will touch. The lead agent maintains a lock table of modules currently claimed by in-flight tickets and refuses to start a ticket whose declared scope overlaps one already claimed; non-overlapping tickets start in parallel without any ordering decision. A coder that discovers mid-ticket that it must touch an undeclared module stops and re-declares, which is an escalation trigger if the module is claimed. Merges go through a merge queue that rebases and re-runs the gates before landing. When one product's ticket volume exceeds what one lead can review, the lead role is partitioned by product area, each instance owning a disjoint set of modules, with a single review queue for humans.

Pairing mode

A human drives a coding agent directly, in a terminal or an agentic coding tool, on one ticket. The human watches the agent work, redirects it when it takes a wrong approach, and co-designs as it goes. Pairing mode is reserved for three cases:

- Decisional tickets and tier 2 changes (authentication, money, data schema, infrastructure)
- Tickets escalated by the lead agent
- New architecture work where the human is co-designing anyway

Pairing is not the default because a human watching one agent per ticket makes the team size scale with the workload, which is exactly what AICD removes.

The escalation protocol

The key design object is neither mode; it is the protocol by which an agent stops and asks. Agents do sometimes get stuck in an approach, and the methodology handles this structurally rather than by hoping a human notices.

BUDGETS AND THE BLOCKED REPORT

Every ticket carries a budget: attempts, wall-clock time, tokens. When any limit is exceeded, the coder stops, writes a blocked report (what it tried, why it believes the approaches fail, what it would try differently, what it needs from a human) and hands off. It never keeps grinding. Blocked reports are written to operational memory so the next attempt starts informed.

LEAD AGENT ESCALATION TRIGGERS

The lead agent must escalate to a human, and may not decide alone, when any of the following occurs: a change touches an area covered by an ADR; a test had to be modified or removed for the build to pass; a new dependency or external service is needed; the coder's plan contradicts the specification; two tickets in progress conflict; a change would alter a public or internal contract; the coder's report contains a security concern. Each escalation is a question addressed to a human, with the lead agent's recommendation attached.

Blocked and escalated items land in a human review queue that is processed at a fixed cadence (for example twice a day). Only incidents interrupt a human outside the cadence. This protects the humans from becoming an always-on bottleneck while keeping the agents from ever making a decision that belongs to a person.

13 Git and release workflow

Git is the spine of AICD. Because agents produce many more changes than a human team, the workflow is stricter, not looser. Every rule below applies equally to agents and humans.

Branches

- `main` is always deployable and protected. No direct pushes, by anyone.
- One branch per ticket, created from `main`: `feat/<ticket>`, `fix/<ticket>`, `ops/<ticket>`, `spec/<ticket>`. An agent works only inside its own branch.
- No long-lived development or release branches. Work that is not ready stays in its branch or ships behind a feature flag.
- Branches are rebased on `main` before the pull request is marked ready, so conflicts are resolved by the coder, not by the reviewer.

Commits

Agents commit often, using Conventional Commits. Every commit message references the ticket and the specification section it implements. This is the audit trail: any line of code can be traced to a commit, to a ticket, to a paragraph of specification, to a human decision.

Pull requests: the control point

Every pull request carries, without exception:

- Link to the ticket and the specification anchor
- The agent's plan, written before implementation, and any deviation from it
- The list of tests added or modified, and why those tests
- The coverage matrix (criterion to test mapping, see section 14)
- What was deliberately not tested, and why
- Self-assessed risk tier
- For tier 2: a written rollback plan
- CI green: lint, type check, full test suite, mutation score threshold, security scan, build

Merge policy by risk tier

Tier	Scope	Merge requirement
------	-------	-------------------

Tier 0	Documentation, logs, copy, dependency patch versions, formatting, test-only additions	CI green and lead agent approval. Auto-merge allowed.
Tier 1	Feature and fix code that does not touch a tier 2 area and does not change a contract	CI green, lead agent approval, one human verifier who reviews the plan, the coverage matrix, the report and the diff summary.
Tier 2	Authentication and authorization, payments and anything touching money, data schema and migrations, infrastructure, secrets, external contracts, anything covered by an ADR	CI green, lead agent approval, two humans, written rollback plan, and at least one human reads the diff itself.

Tags, releases and rollback

- Every deployment is made from a tag on main. The changelog is generated from the commits.
- Rollback is always "redeploy the previous tag". There is no hotfix directly on main; a hotfix is a fix ticket that goes through the normal flow with an elevated priority.
- Database migrations are written to be backward compatible with the previous tag, so a rollback never requires a data rollback.

14 Verification: tests written in language, built by AI, checked by humans

In a system where humans do not read most of the code, the test suite and the observability stack are the only things that prove the software does what the specification says. Verification is therefore where human time goes in AICD, and the methodology is careful about where in the process that time is spent.

The problem verification must solve

A human who does not read test code cannot judge a test suite by reading it. So the human verification must happen one level up: on the mapping between what the specification demands and what the tests prove, rather than on the tests themselves. This is what makes it possible for a verifier with deep system design knowledge to sign off on a suite of tests without reading a line of them, and makes it impossible for an agent to quietly weaken a test without a visible signal.

The verification cycle

1. **Acceptance criteria (human, with AI assistance).** Before any build, the verification lead writes the acceptance criteria for the ticket in plain language, working with the assistant agent to find edge cases and failure modes. Each criterion has an identifier, a precondition, an action, an expected observable result, and a type (functional, security, performance, resilience, simulation). This structure matters: free prose is fine for thinking, but the final artifact must be structured enough for a machine to check the mapping to tests.
2. **Test plan (QA agent).** The QA agent produces a test plan: a table with one row per criterion, the test type that will cover it (unit, integration, end-to-end, security, load, chaos, simulation), and the name of the test that will be written. No code exists yet.
3. **Test plan verification (human).** The verification lead checks the plan: is every criterion covered, are the edge cases present, is anything declared untestable, and is the level appropriate. An authorization rule covered only by a unit test is a red flag; a payment flow with no end-to-end test is a rejection.
4. **Build (coder agent).** The coder writes tests and code, in the project's language, from the plan. CI enforces the mapping: every criterion identifier must appear in at least one test, and every test must map to a criterion, or it is flagged.
5. **Evidence review (human, at pull request).** The verifier reads the coverage matrix, the CI report, the mutation score, and the agent's list of what it did not test and why. Mutation testing runs in CI so the team knows the tests actually fail when the code is broken, not only that they pass when it works.

RULE: MODIFIED TESTS ALWAYS ESCALATE

Any pull request in which an existing test was modified, weakened or removed triggers an escalation to a human, whatever the tier. Passing tests by changing tests is the most common way an agent hides a regression, and the methodology treats it as such.

RULE: A GATE IS INSTALLED ONLY WHEN IT HAS BEEN SEEN TO FAIL

A gate that has only ever been observed passing is not installed. Every gate (a CI check, a coverage matrix checker, a mutation threshold, a liveness check, a permission rule) enters service only after a demonstration on a planted defect: it passes on a clean tree, it fails on the planted defect, and the failure is visible where a human would look. The demonstration is recorded in the ticket that installed the gate. The recurring defect class this rule exists for is "present but reporting nothing": a checker that exits successfully on every input, a workflow file whose trigger never fires, a test stage that a deploy path bypasses. Each of these reads as protection in every document that cites it while protecting nothing.

Test types in AICD

AICD does not limit itself to the classical pyramid. All of the following are part of the standard test plan vocabulary, and the QA agent is expected to propose the right mix for each ticket:

Type	What it proves
Unit	Individual functions and modules behave as specified in isolation
Integration	Components work together across their real interfaces (database, queues, external APIs with contract stubs)
End-to-end	Complete user journeys succeed through the real interface
Security	Static analysis, dependency audit, dynamic scanning, and periodic AI-driven penetration attempts within a defined scope
Load and performance	The system meets its non-functional targets under load profiles derived from real usage
Chaos and resilience	The system degrades gracefully when a service dies, the database slows, a token expires or a dependency times out
Simulation	Synthetic users with personas modeled on real segments use the product in many parallel runs, revealing behaviors no scripted test would
Property-based and fuzzing	Invariants hold across generated inputs; parsers and interfaces survive malformed data
Contract	Public and internal interfaces have not changed in ways that break consumers
Mutation	The test suite itself is effective: introduced faults are detected

15 The continuous test environment

A human QA team tests when it is asked to. The AICD QA agent tests all the time. Each product has a persistent staging environment that mirrors production infrastructure, and the QA agent lives in it.

What runs there

- Seeded synthetic data and anonymized replay of production traffic
- End-to-end journeys, on every deploy and on a schedule
- Load profiles derived from real product analytics, so the load test reflects how users actually behave
- Security scanning and periodic penetration attempts, scoped and logged
- Chaos experiments: killing services, slowing the database, expiring credentials, cutting a third-party dependency
- Simulations with synthetic user personas that behave like the product's real segments

Cadence: event-driven, not permanent

"Continuous" means the environment is always available and always current, not that every test runs all the time. Simulation, load and chaos runs are expensive and produce the same findings if repeated on an unchanged system, so they are triggered by significant modifications rather than by the clock. A run is exhaustive: the QA agent launches as many simulations in parallel as needed, collects every issue it can find, files them, and then stops until the next trigger. The scheduled part of the environment is limited to cheap checks (health, smoke journeys, dependency and security scans).

Test family	Triggered by
Smoke and end-to-end journeys	Every deploy to staging, every deploy to production
Security scanning	Every pull request (static), every deploy (dynamic), and a scheduled dependency audit
Load and performance	Any change to a hot path, a data access pattern, infrastructure, or a non-functional target; any new endpoint
Chaos and resilience	Any change to infrastructure, to a runbook, to a third-party dependency, or to error handling
Simulation with personas	Any significant modification (see below); never for cosmetic changes
Penetration attempts	Any tier 2 change and a scheduled periodic run

A **significant modification** is any merged change that alters a user journey, a business rule, a data model, a contract, a permission, an integration, or a non-functional characteristic. Changes to copy, styling, layout, logs, documentation or dependency patch versions are not significant. The lead agent marks the significance on each merged pull request from the ticket category and the affected modules, and the QA agent uses that mark as its trigger. The list of what counts as significant is part of organizational knowledge and can be extended per product.

What comes out

Every finding becomes a ticket with a proposed category, severity, reproduction steps and evidence. Findings that reveal a gap in the acceptance criteria also produce a *proposed* criterion, so that the gap is closed permanently rather than fixed once.

Proposed criteria never enter the coverage matrix on their own. They sit in a proposed state until the verification lead accepts, rewrites or rejects them, and the acceptance rate is a calibration metric (section 30). This is what prevents the QA agent from inflating the criteria set with low-value edge cases that then drive build-and-test loops nobody asked for. Each QA run also carries a budget (parallel runs, wall-clock, tokens), set by the performance-relative procedure of section 21, and stops when the budget is reached or when findings saturate, whichever comes first.

Constraints

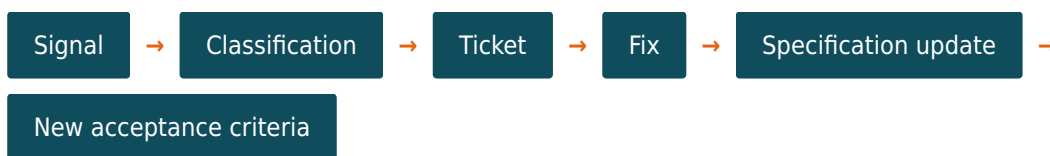
- The QA agent reads production telemetry but never writes to production and never writes to code repositories. Its only outputs are tickets and staging state.
- The environment is probably the largest infrastructure cost line of the methodology. It scales down outside test windows and its cost is tracked per product (section 21).
- Simulation and penetration activity is scoped in writing, so that an agent exploring aggressively never leaves the environment it was given.

16 Operations and the observability loop

Production is watched continuously through error tracking, product analytics, logs, metrics, uptime and cost. Three agents watch three different things:

- The **operations agent** watches health: anything crossing a service level objective is an incident. It executes runbooks within its authorization (restart, scale, roll back to the previous tag) and opens an incident ticket.
- The **QA agent** watches correctness: errors, exceptions and failed journeys become defect tickets with evidence attached.
- The **product signal agent** watches usage: patterns in analytics become product signal tickets for the product owner.

The loop



The last two steps are what distinguish AICD from "AI fixes bugs". A fix without a specification update produces specification rot; a fix without new acceptance criteria produces a bug that can return. Both steps are required for a ticket to close.

Incidents and post-mortems

Incidents are the one case where a human is interrupted outside the review cadence. The operations agent handles the immediate response within its runbooks; a human owns the decision to do anything not covered by a runbook. Every incident gets a post-mortem, written with the AI but owned by a human, recorded in operational memory, and translated into acceptance criteria, runbook updates or ADRs as appropriate.

RULE: PRODUCTION DATA IS UNTRUSTED INPUT

Anything an agent reads from production, including error messages, user-generated content, log lines and analytics events, is data, never instructions. A malicious user can trigger an error whose message contains text designed to steer an agent; the methodology assumes this will happen and enforces the rule structurally, in how tool results are presented to agents, not by asking the agent to be careful.

17 Autonomy tiers and the permission model

An agent with connections to the error tracker, the analytics platform, the database and the deployment pipeline is the most privileged entity in the company. AICD treats agent permissions as a security architecture, enforced at the infrastructure and tool-connection level, never as a line in a prompt.

Principles

- Every agent has its own identity and its own short-lived, scoped credentials. No agent ever holds a credential with more rights than its role requires.
- No agent holds production write access to code. Code changes exist only as branches and pull requests.
- The coder agents never see production at all: no production data, no production credentials, no production telemetry.
- Every agent action is written to an immutable audit trail: which agent, which ticket, which tool, what input, what output, when.
- Humans hold the permissions that agents do not: tier 1 and 2 merges, permission changes, runbook approval, and anything not explicitly granted.

Permission matrix

Agent	Code repositories	CI	Staging	Production	Tickets	Specification
Coder	Read; write own branch	Run	Deploy own branch	None	Read own; comment	Read

Lead / Reviewer	Read all; merge tier 0	Read	Read	None	Read, assign, upgrade category	Read
QA	None	Read	Read, write	Read telemetry only	Create, read	Read; propose criteria
Operations	None	Read	Read, write	Deploy from tags, rollback, runbooks	Create incident, read	Read runbooks
Documentation	Read; write spec branches	None	None	None	Read	Propose via pull request
Product signal	None	None	None	Read analytics only	Create product signal	Read product brief
Assistant	Read	None	None	None	Read	Draft only

Autonomy tiers in practice

The risk tiers of section 13 define what an agent may merge; the ticket categories of section 11 define what it may start; the permission matrix defines what it may touch. Together they form the autonomy model. A new project starts with the strictest settings and loosens them only with evidence, and the loosening is itself a governance decision recorded in an ADR. Increasing model capability lowers the cost of running the tiers; it does not remove them.

PART IV

People and organization

Who the humans in an AICD team are, what they own, how they are hired, and how they stay capable of disagreeing with the machines they direct.

18 Human roles

An AICD team is small, and every member is an experienced developer with a background in system design. Nobody writes code; everybody can read it, in any language, and judge it. The team is organized in three seats plus a product owner. Early on, one person may hold more than one seat; the seats stay distinct because their responsibilities and permissions are distinct.

Seat	Owns	Approves
Architect / Orchestrator	Canonical knowledge: architecture, ADRs, domain model, contracts, conventions, agent instructions. Stack selection. Escalations from the lead agent. Co-design of new features with the assistant agent. Configuration of the coder and lead agents.	Decisional tickets. Tier 2 pull requests (with a second human). Specification pull requests.
Verification lead	Acceptance criteria. Test plans. Configuration of the QA agent and the continuous test environment. Coverage and mutation thresholds. Post-incident acceptance criteria.	Test plans before build. Tier 1 pull requests. Ticket category downgrades.

Reliability and governance	Agent identities and permissions. Secrets. Infrastructure and deployment topology. Runbooks. Incident response and post-mortems. Cost tracking. The audit trail. Configuration of the operations agent.	Tier 2 pull requests (with the architect). Runbook changes. Permission changes. Autonomy loosening.
Product owner	Product brief. Prioritization. Product signal tickets. The decision of what is worth building.	Feature tickets entering the specify step. Scope changes.

The four human functions, across seats

- **Supervisor:** watches the fleet, the queues, the costs and production; intervenes on drift.
- **Controller:** verifies test plans and evidence; approves according to tier; owns the audit trail.
- **Co-architect:** designs the system and each new feature together with the AI, and decides.
- **Product owner:** decides what should exist and why.

19 Hiring and keeping judgment sharp

Hiring criteria

AICD asks for a rarer profile than "developer": someone who has built and operated real systems, and who is comfortable never typing the implementation again. The criteria:

- **System design experience.** Has designed, shipped and lived with production architectures; understands trade-offs in data modeling, consistency, scaling, deployment and failure.
- **Reads code in any language.** Can open a diff in a language they have never written in and spot a dropped table, a removed authorization check, a hard-coded secret or a suspicious dependency.
- **Writes precisely.** Specification and acceptance criteria are now the core craft. Vague writers produce vague systems.
- **Disagrees with confident answers.** The agents are fluent and persuasive. The team needs people whose instinct on a polished proposal is "prove it".
- **Operational maturity.** Comfortable with observability, incidents and the reality that the system runs while they sleep.

Skill atrophy and how AICD answers it

A verifier can only judge system design if their judgment stays sharp, and judgment is usually built by building. If nobody in the team has written code for years, will they still detect a bad architecture proposed by the AI? AICD does not answer this by letting people code. It answers it with deliberate practice built into the methodology:

- **Monthly architecture reviews without AI.** Humans critique a recent AI-produced design or a significant pull request with no AI assistance, then compare their conclusions with the agents'. The gaps, in both directions, are recorded.
- **Human-owned post-mortems.** Every incident is analyzed by a human, not summarized by an agent. Understanding failures is where design judgment comes from.
- **Rotating review.** Verifiers rotate across products so that nobody's understanding narrows to one system.
- **Tier 2 diff reading.** The requirement that a human reads the actual diff on every tier 2 change is a control, and it is also the practice that keeps code literacy alive.

20 Stack selection policy

Before AI coding agents, teams chose technologies partly on the availability of people who knew them. That constraint is gone. An agent works in any language and framework that has good documentation, which means technologies that were long avoided for their complexity or their specialized nature (functional languages, actor systems, systems languages, heavyweight enterprise frameworks) are now practical choices whenever they are the best fit.

AICD therefore has no default stack. It has a selection policy:

- The stack is chosen per project, for the result it delivers on that project's requirements, never for popularity.
- Eligible technologies are mature, production-proven for years, and well documented. Novelty is not a disqualifier by itself, but an undocumented or unproven technology is.
- Every selection is recorded in a stack ADR: requirements, options considered, why this one, what it costs in operations, and what would make the team revisit it.
- Per-project freedom is balanced by cross-product conventions in organizational knowledge (how deployments, observability, secrets and the agent roles work), so the team's cognitive load stays on design rather than on tooling differences.

PART V

Governance, economics, resilience

What it costs to run AICD, how to know whether it is working, what can go wrong, and what remains to be explored before the methodology is complete.

21 Economics and metrics

AICD replaces most salary cost with token cost plus infrastructure, and turns both into quantities that can be measured per unit of work. This is a significant advantage over traditional teams, where the cost of a feature was an estimate.

WHY AICD GIVES NO ABSOLUTE COST BASELINE

Model prices change constantly, and a more capable model consumes fewer tokens for the same ticket because it succeeds in one pass instead of several. Any absolute figure written into a methodology would be wrong within months. AICD therefore defines the quantities to measure and the way to set thresholds, and leaves the numbers to each team, to be re-measured whenever the model changes.

Performance-relative thresholds

Every budget and threshold in AICD (tokens per ticket, attempts, wall-clock time) is expressed relative to the measured performance of the models in use, not as a fixed number. The procedure:

1. Maintain a **calibration set**: a small number of representative closed tickets per category (Auto, Behavioral, Decisional), with known correct outcomes.
2. When a model is adopted or upgraded, run the calibration set through the fleet and record, per category: first-pass success rate, median tokens to completion, median attempts, median time.
3. Set budgets as multiples of the medians (for example three times the median tokens, twice the median attempts). The multiple is the team's tolerance; the base is the model's measured performance.
4. Re-run the calibration whenever the model, the agent instructions or the memory service changes materially. A budget that was right for one model is wrong for the next.

The same logic applies to escalation and categorization thresholds (section 30).

The methodology requires that the following be tracked, per product and per month:

Metric	What it tells you
Tokens and cost per ticket, by category	Which kinds of work are expensive; whether Auto tickets are as cheap as they should be
Cost per merged pull request	The unit cost of change, including failed attempts and reviews
Continuous test environment cost	Usually the largest infrastructure line; whether it scales down correctly
Memory service cost	Indexing and retrieval; grows with the number of products
Human hours per approved ticket, by category and tier	Whether humans are still the bottleneck, and where
Escalation rate	Too high and humans drown; too low and architecture drifts unreviewed
Blocked rate and average attempts to completion	Whether budgets are tuned correctly and whether specifications are clear enough
Time from ticket validated to deployed	The flow speed the methodology promises
Defects found in production versus in the test environment	Whether continuous testing is doing its job
Mutation score and coverage matrix completeness	Whether the verification layer is actually proving things

Expect coder agents to be the cheap part, and continuous QA plus memory indexing to be the parts that surprise. The economics of a project are judged on these numbers, not on intuition.

22 Failure modes and safeguards

Every methodology fails in characteristic ways. AICD names its own in advance and builds the answer into the structure.

Failure mode	What it looks like	Safeguard
Model provider outage	Development stops because the agents cannot run	Products never depend on AI at runtime for their own operation; only the development pipeline does. The agent layer is provider-agnostic so coders can run on another model family with degraded quality. Production keeps running regardless.
Agent looping	A coder retries the same failing approach indefinitely, burning budget	Per-ticket budgets and the blocked report protocol (section 12).
Silent test weakening	An agent makes the build pass by changing or removing a test	Mutation testing in CI; the rule that any modified test escalates to a human.
Specification rot	Docs no longer describe the system; since docs are the source of truth, the methodology silently breaks	Documentation agent on every merge; weekly drift audit; "last verified against code" dates; tickets cannot close without a specification update.
Correlated blind spots	Coder and reviewer share the same model and miss the same class of error	Lead/reviewer on a different model family; a human reads every tier 2 diff.
Credential leakage	A key on an agent's machine is compromised and grants wide access	Short-lived, scoped, per-agent credentials; coders never see production; immutable audit trail.
Prompt injection via production data	Text in an error message or user content steers the monitoring or QA agent	Production data is untrusted input, enforced structurally in how tool results reach agents (section 16).
Human bottleneck	A QA agent running around the clock files more tickets than humans can approve	Ticket categories with automatic validation for Auto and Behavioral; review cadence; upgrade-only rule.
Unreviewed architectural drift	Many small Auto and Behavioral tickets accumulate into a de facto architecture change	ADR-covered areas always escalate; monthly human architecture review; the lead agent's contract-change trigger.

Parallel conflicts	Two coders modify the same module and produce incompatible pull requests	Lead agent orders the queue by affected modules; rebase before ready; conflicting tickets are an escalation trigger.
Knowledge concentration	The architect leaves and nobody can operate the system	Complete canonical documentation is a requirement, not a nicety; a new architect should be operational within weeks from the documents alone.
Human skill atrophy	Verifiers lose the judgment needed to disagree with the agents	Deliberate practice built into the methodology (section 19).

PART VI

Applying AICD

How a product enters the methodology: either born inside it, or migrated into it from a codebase that already exists. Both paths end in the same steady state described in Parts II and III; they differ in what has to exist before the agent fleet is allowed to work.

23 Starting a new product under AICD

A new product under AICD follows a fixed sequence of gates. Nothing is built before the specification exists; no feature is added without a plan; no human touches the code. The sequence comes from the Vibe Engineer process that preceded AICD (framing session, thirteen documents, phased build loop, three golden rules) and extends it with the agent team, the verification layer and the governance of Parts II and III.

THE THREE GOLDEN RULES, INHERITED

Never skip the documents. They are what stops the AI from improvising a different decision at every phase, and they are the human's version of the code.

Never add a feature without a plan. Before any new work the agent analyzes what exists, explains how the new thing integrates and what it touches, and builds only after.

Never touch the code. Every change is requested in language. A manual edit puts the system outside what the agents know.

The gates



G0: Framing session

The product owner and the architect run a framing session with the assistant agent. The goal is to turn a vague idea into a bounded project. The agent is explicitly asked to challenge the idea: who really has this problem, what already solves it, what makes this different, what would make it fail. The session ends with a written scope that includes the non-goals, meaning what the product will not do, and a validation grid answered honestly. Exit criterion: the idea can be stated in one paragraph, the target user in one sentence, success in one measurable statement, and the non-goals in a list.

G1: Specification

The assistant agent generates the specification set from the framing output. The product brief is generated first and verified line by line by the humans, because everything else derives from it; the remaining documents are generated only once the brief is correct. AICD's specification set for a new product:

Document	Verified by	What it fixes
PROJECT_BRIEF	Product owner, architect	Problem, user, definition of success, non-goals. The only document read in full by every human before anything else exists.
PRD	Product owner	Features by priority, user journeys. The "what".
ARCHITECTURE	Architect	Components and how they communicate; deployment topology; update strategy. Prevents the agents from changing tools mid-project.
Stack ADR	Architect	Requirements, options considered, the chosen stack and why, operational cost, revisit conditions (section 20).
DATA_MODEL	Architect	Entities, relationships, invariants, state machines, in text and Mermaid.
API_SPEC	Architect	Interfaces between front and back, between services, and with third parties; error semantics.
LLD	Architect	Project layout, responsibilities of each part, naming.
CONVENTIONS	Architect	Coding rules, commit format, branch rules, what agents always and never do.

SECURITY_NOTES	Architect, reliability and governance	Who may do what, how input is validated, how edge cases are handled, tenant isolation. Defines which modules are tier 2.
ENV_SETUP and permission manifest	Reliability and governance	Every credential the product needs, and, new in AICD, which agent identity gets which credential at which scope.
TESTING and acceptance criteria	Verification lead	Test strategy per level, coverage and mutation thresholds, and the first acceptance criteria per phase of the roadmap.
OBSERVABILITY	Reliability and governance	What is measured, the service level objectives, the alert rules, which agent watches what.
CI_CD and runbooks	Reliability and governance	How a validated change reaches production; deploy, rollback, scale and recover procedures the operations agent may execute.
ROADMAP	Product owner, architect	Build phases, each small enough to be one batch of tickets with its own acceptance criteria.
Agent instructions	Architect	The file every agent keeps open: a summary of the others plus the absolute rules. In AICD there is one per agent role, derived from a shared base.

Foundation set and phase set

Not every document above has to be complete before the first line of code, and demanding that would recreate the failure of upfront design: certainty on paper about things no one has observed yet. AICD splits the set in two.

- **Foundation set, complete before any code:** PROJECT_BRIEF, ARCHITECTURE, the stack ADR, DATA_MODEL at the level of entities and invariants, SECURITY_NOTES with the tier 2 modules, ENV_SETUP with the permission manifest, CONVENTIONS, OBSERVABILITY, CI_CD, ROADMAP, and the agent instructions. These fix what is expensive to change later, and agents cannot be allowed to improvise them.
- **Phase set, completed per roadmap phase, before that phase starts:** the PRD detail for the phase's features, API_SPEC and DATA_MODEL detail for what the phase touches, LLD for the modules it creates, the acceptance criteria and test plan for the phase, and the design artifacts for its journeys. They are written when the previous phase has produced real behavior to learn from.

The rule that nothing is built outside the specification is unchanged: a phase's specification exists before the phase is built. What changes is that detail is written just in time, at the granularity of a phase, not for the whole product on day one.

Exit criterion for G1: every foundation document has an owner's sign-off, the security notes name the tier 2 modules, the permission manifest is complete, and the phase set for the first roadmap phase is complete.

G2: Design

Screens, flows and visual language are produced with a design agent or tool from the PRD, and validated by the product owner before any code. Design artifacts are referenced from the PRD and become inputs to the coder agents. Exit criterion: every user journey in the PRD has a validated design.

G3: Environments and access

Accounts, infrastructure, repositories, the continuous test environment, observability tools and third-party services are created, and access is attributed from the permission manifest. Humans receive their seat permissions first; then each agent identity is created with short-lived, scoped credentials and nothing more. No agent ever receives a human's personal key. Exit criterion: the permission matrix of section 17 is enforced by the tools, verified by attempting one forbidden action per agent and confirming it fails.

G4: Fleet setup

The agent roles of section 7 are instantiated for this product with their instruction files, their memory scopes, and the lead agent on a different model family than the coders. The memory service is loaded with the specification (layer 1) and the organizational knowledge (layer 2). The ticketing system, the pull request template and the CI gates (lint, types, tests, coverage matrix, mutation score, security scan) are in place. Exit criterion: a deliberately trivial tier 0 ticket travels the full path from queue to merged pull request without human intervention, and a deliberately forbidden action by a coder is refused.

G5: Phased build

The roadmap is executed phase by phase. A phase is a batch of tickets, validated by the humans, worked in fleet mode. Before each phase the verification lead confirms the acceptance criteria and the QA agent's test plan; during the phase the lead agent reviews and escalates; at the end of the phase the humans explore the product themselves on staging and give feedback in product language, which becomes tickets. A phase closes when its acceptance criteria are all covered, the coverage matrix is complete, and the QA agent's exhaustive run on staging reports no open Auto or Behavioral tickets. Exit criterion for G5: every roadmap phase is closed.

G6: Go-live

A tag is cut, deployed to staging, and the full QA run (end-to-end, security, load, chaos, simulation) executes against it. The same tag is then promoted to production; there is never a separate build for production. Observability is confirmed live: the operations agent sees production, the product signal agent sees analytics, alert rules fire in a test. Exit criterion: the rollback runbook has been executed once for real, successfully, before the first users arrive.

G7: Steady state

The product is now in the flow of Part III. The fleet works validated tickets; the QA agent runs on significant modifications; the documentation agent keeps the specification current; humans supervise, verify and decide. The first month runs with the strictest tiers; loosening follows the evidence rule of section 17.

BRANCHES FOR A NEW PRODUCT

The solo Vibe Engineer process used three long-lived branches (dev, staging, main) because one person and one agent needed a simple safety ladder. In AICD the ladder is provided by environments and tag promotion, not by branches: the fleet works on many short-lived ticket branches, merges to a single protected main, and the same tag is deployed to staging and then promoted to production. This gives the same safety with fewer merge conflicts and a clean audit trail. Teams migrating from the three-branch model keep their environments and drop the two extra branches.

24 Migrating an existing product into AICD

This is the harder path, and the one most teams will actually take: a product already exists, already has users, and was built either by vibe coding (with or without documents), by a traditional team, or by a mix. The agents cannot be trusted with it until they know it, and the humans cannot verify agent work on it until they have a baseline of what it does today. Migration is therefore mostly a documentation and access exercise, done in a strict order, with the fleet kept out until the last step.

THE MIGRATION RULE

No behavioral change is made to a product under migration until three things exist: an as-built specification verified against the code, a verification baseline that captures current behavior, and an access attribution in which no agent holds anything beyond its role. Until then the only permitted changes are the migration's own artifacts.

Three origins, one path

Origin	What it means for the migration
Vibe-coded with generated documents	The best case, but the documents were written before the code and have drifted. They are treated as the intended specification, not the as-built one; the first job is a drift audit that produces the divergence register.
Vibe-coded without documents	The as-built specification is reverse-engineered entirely from code, telemetry and the product owner. Expect the largest divergence between what the owner thinks the product does and what it does.
Built by a traditional team	Usually partial documentation, real tests of uneven value, and people who hold knowledge in their heads. Interviews are a source. The human transition (section 24.8) is the delicate part.

24.1 Phase M0: Freeze and snapshot

- Declare the migration. From this moment no human edits code by hand, on any branch, for any reason. If a production emergency requires a change, it is done as a fix ticket in pairing mode, recorded as such.
- Tag the current production state as the **migration baseline**. Every later comparison is against this tag.
- Inventory everything the product is made of: repositories, branches with unmerged work, environments, services, databases and their sizes, queues, scheduled jobs, third-party services, domains, certificates, and every place a credential lives.
- Inventory every human who currently has access to anything, with what rights.

Exit criterion: the inventory is complete enough that a new engineer could draw the system from it. Unknowns are listed as unknowns, not omitted.

24.2 Phase M1: Access attribution

Access is attributed before any agent reads the codebase, because the agents that will document the system need read access, and giving them anything more at this stage is the classic mistake. Order of operations:

1. **Humans first.** The seats of section 18 are assigned. Personal accounts with broad rights are replaced by seat-scoped roles. Anyone not in a seat loses write access to production.
2. **Credential rotation.** Every credential in the inventory is rotated. Anything that was ever pasted into a chat, a prompt, a notebook or an agent's configuration is assumed compromised.
3. **Agent identities.** One identity per agent role, with short-lived credentials. No agent reuses a human's key, and no agent shares an identity with another agent.
4. **Migration-time permissions.** Stricter than steady state, because the specification does not exist yet.

Agent	During migration	After migration (steady state, section 17)
Documentation	Read all code and infrastructure definitions; write only to the migration documentation branch	Read; propose specification pull requests
Assistant	Read all; draft only	Unchanged
QA	Read code (exception, for characterization only); read production telemetry; write staging once it exists	No code access; staging and tickets
Operations	Read production telemetry and infrastructure; no deploy, no runbook execution	Deploy from tags, rollback, runbooks
Product signal	Read analytics	Unchanged
Lead / Reviewer	Not instantiated	Full role
Coder	Not instantiated	Own branch, CI, staging

Exit criterion: no human holds a personal production key; no agent holds anything beyond the migration column; every forbidden action has been attempted once and refused.

24.3 Phase M2: Reverse-engineering the as-built specification

This is the core of the migration. The documentation agent, driven by the architect, produces a set of as-built documents from the code, the infrastructure, the telemetry and interviews with the people who know the product. Each document is generated by the agent and verified by the human seat that owns it. The order matters: later documents depend on earlier ones.

#	Document	Content	Source and verification
1	System inventory and as-built architecture	Components, services, data stores, queues, jobs, external integrations, deployment topology, how requests flow	Generated from code and infrastructure definitions; verified by the architect against the running system
2	Dependency and third-party register	Every library with version and license; every external service with what it is used for, the credential involved and the failure impact	Generated from manifests and code; verified by reliability and governance
3	As-built domain model	Entities, relationships and invariants as they actually exist in the schema and the code, including the ones that contradict each other	Generated from schema and code; verified by the architect
4	Data inventory and classification	Every table or collection, its size, whether it holds personal data, financial data, credentials or content, retention, and who can read it	Generated from schema; classification decided by reliability and governance. Drives anonymization for the test environment.
5	As-built interface contracts	Every endpoint, event, webhook and job interface with its real inputs, outputs and error behavior, including undocumented ones	Generated from code and traffic; verified by the architect
6	As-built security map	Authentication and authorization as implemented; where input is validated and where it is not; secrets handling; tenant isolation; known exposure	Generated from code and a full security scan; verified by reliability and governance. Names the tier 2 modules.
7	As-built operations	How the product is deployed today, how it has been rolled back, backups and their last restore test, monitoring already in place	Generated from pipelines and infrastructure; verified by reliability and governance. Becomes the first runbooks.

8	Known issues and debt register	Everything the audit finds: dead code, duplicated logic, unsafe patterns, missing validation, flaky behavior, performance hot spots	Generated by agent audit; prioritized by the architect. Feeds the first migration tickets.
9	Product brief and PRD, reconstructed	What the product is for, for whom, the features that exist and their intended behavior	Interviews with the product owner, analytics, the existing documents if any; written by the product owner with the assistant agent
10	Divergence register	Every place where the intended behavior (9, and any pre-existing documents) differs from the as-built behavior (1 to 8)	Produced by the drift audit; each entry is resolved by a human decision: the code is right and the intent updates, or the intent is right and a ticket is created
11	Inherited ADRs	The significant decisions already embedded in the code (stack, data store, auth approach, integration patterns), recorded with status "inherited", with their consequences and their escalation scope	Drafted by the documentation agent; accepted or marked for revisit by the architect
12	Observed conventions and target conventions	The rules the code actually follows, and the rules it should follow from now on; the gap is a debt item, not an excuse to rewrite	Generated; decided by the architect
13	Risk map	Every module classified into tier 0, 1 or 2, with the reason	Derived from 4, 6 and 8; decided jointly by the architect and reliability and governance
14	Agent instructions	One file per agent role for this product, derived from the shared base and the documents above	Written by the architect with the assistant agent

AS-BUILT FIRST, INTENDED SECOND

The temptation during migration is to write the specification the product should have had. AICD requires documenting what exists first, exactly, including its flaws, and only then deciding what should change. A specification that describes an imagined product is worse than none: the agents will "fix" the real product toward it without anyone having decided that, and the verification baseline (M3) will not match either.

Exit criterion: every document has its owner's sign-off, every divergence entry has a human decision, and the risk map covers every module.

24.4 Phase M3: Verification baseline

Before any agent is allowed to change the product, the team needs to know what "unchanged" means. The baseline captures current behavior so that every later change can be shown to preserve it or to alter it deliberately.

- **Characterization criteria.** The verification lead, with the QA agent, writes acceptance criteria that describe current behavior as observed, journey by journey, including behavior the team considers wrong (those criteria are marked "to change" and linked to divergence entries). These are the first entries of the product's criteria set.
- **Existing tests audit.** Every existing test is mapped to a criterion or flagged as unmapped. Unmapped tests are kept but marked; they are not deleted until a criterion covers what they were protecting.
- **Baseline measurements.** Mutation score, coverage matrix completeness, security scan results, performance under a load profile derived from real analytics. These numbers are recorded against the baseline tag; the methodology's thresholds start from them, not from an ideal.
- **Continuous test environment.** Stood up from the as-built operations document, seeded with anonymized data according to the data classification. The QA agent runs a first exhaustive pass and files everything it finds; those findings join the debt register.

Exit criterion: every user journey in the reconstructed PRD has characterization criteria; the coverage matrix exists, even if sparse; the baseline numbers are recorded; the QA agent's first exhaustive run is complete.

24.5 Phase M4: Observability onboarding

Error tracking, analytics, logs, metrics and uptime are connected if they were not, service level objectives are set from the baseline measurements, and alert rules are written. The operations agent and the product signal agent go live in read-only mode: they classify and file tickets but execute nothing. The team watches the ticket stream for a fixed period to calibrate categorization (section 30) before any of it becomes actionable.

24.6 Phase M5: Fleet activation and progressive autonomy

Only now are the lead agent and the coder agents instantiated, with the memory service loaded with the as-built specification, and given the steady-state permissions of section 17. Autonomy is opened in steps, each gated by evidence:

Step	What the fleet may do	Evidence to move on
Shadow	Work Auto tickets from the debt register; every pull request treated as tier 2 (two humans, one reads the diff)	A defined number of pull requests merged with no human-found defect; coverage matrix growing
Tier discipline	Normal tiers apply, but auto-merge is disabled; Behavioral tickets allowed	Escalation rate and blocked rate within the calibrated range; mutation score not decreasing

Steady state	Full Part III flow, including tier 0 auto-merge and operations agent runbook execution	One full cycle of significant modification, QA run, specification update and observed production behavior without incident
--------------	--	--

The step durations are set by the team and recorded; they are not part of the methodology, because they depend on the product and the models.

24.7 Migration exit checklist

- No human has edited code since M0, or every exception is recorded as a ticket.
- The as-built specification is complete and signed off; the divergence register is fully resolved.
- Every module has a tier; every tier 2 module has an ADR or a security note covering it.
- Every credential has been rotated once; every agent has its own scoped identity; the forbidden-action test passes for every agent.
- Characterization criteria cover every journey; the coverage matrix and baseline numbers exist.
- The continuous test environment runs with anonymized data.
- The rollback runbook has been executed successfully by the operations agent on staging.
- The fleet has completed the three autonomy steps.
- The documentation agent's first weekly drift audit reports zero divergence.

24.8 Special cases

A product with a team of coders. The developers stop writing code at M0 and take the seats of section 18 or leave the product. There is no hybrid period in which some code is written by hand and some by agents; hybrid codebases are the failure mode AICD exists to prevent. The developers' knowledge is captured during M2 through interviews, and their judgment is what makes them good verifiers.

Several products sharing a codebase. The shared code is documented once as its own component with its own tier map and ADRs; each product references it. A change to the shared component is decisional for every product that uses it.

A product with no tests at all. M3 is longer, and the shadow step of M5 stays until the coverage matrix reaches the threshold the verification lead sets from the baseline. The temptation to let coders "add tests everywhere" first is resisted: tests without criteria prove nothing, and the criteria come first.

A product whose stack the team would not have chosen. The stack is inherited and recorded in an inherited ADR with revisit conditions. Migration never includes a rewrite. A rewrite, if ever justified, is a new product under section 23 with the migrated product as the reference system.

PART VII

Deep dives

The remaining aspects of the methodology in depth: how the memory service works, how several products are run by one team, the security model of the agent fleet, what the humans see, how the methodology itself evolves, and how its thresholds are calibrated.

25 Memory service: implementation model

Section 8 defined what memory contains and who may see it. This section defines how a memory service is built so that the definition holds in practice. The design is tool-agnostic: any implementation that satisfies these components and rules is an AICD memory service.

Components

Component	Responsibility
Repository indexer	Watches layer 1 and layer 2 repositories. On every merge to main, re-indexes the changed documents: full-text index, semantic embeddings per section, and the document graph (which section references which ADR, which criterion, which module).

Code map builder	Builds and maintains a structural map of each code repository: modules, their public interfaces, dependency edges, entry points, the tests that cover each module, and the specification sections each module implements (from commit and pull request anchors). Updated on every merge.
Operational log store	Append-only store for layer 3: tickets, outcomes, escalation decisions, blocked reports, incidents, post-mortems, closing reports. Every record carries the agent identity, the ticket, the product and the timestamp. Indexed by module and by criterion.
Scope enforcer	Maps each agent identity to a role and each role to a set of readable sources and filters. Enforced at query time; an agent cannot request outside its scope, and the refusal is logged.
Retrieval API	Answers task-oriented queries ("context for ticket 482", "history of module X", "criteria touching payments") by assembling a bounded context package: the relevant specification sections, ADRs with escalation scope, related past tickets and blocked reports, the code map of the affected modules, and the criteria involved. Returns provenance for every item.
Freshness tracker	Records for each indexed document the commit it was indexed at and the "last verified against code" date; exposes staleness so the drift audit and the agents can see when a source may be out of date.

Rules

- The repositories are the truth; the index is a derived cache and can be rebuilt from scratch at any time. Nothing exists only in the index.
- Context packages are bounded. The service returns what the task needs, with provenance, not the whole product. An agent that needs more asks a second question; the service never pre-loads "everything, just in case".
- Every item returned carries its source, its commit and its verification date, so that the agent and, later, a human reviewer can tell where a belief came from.
- Agents write to layer 3 only through structured records (closing report, blocked report, escalation decision), never as free notes. Free notes rot; structured records can be queried and audited. Records derived from production pass through the sanitization barrier of section 8 before they are written.
- Retrieval quality is measured. A calibration set of questions with known correct context (section 30) is run whenever the indexer, the embeddings or the code map builder change.

Multi-product layout

One memory service serves the whole organization. Layer 2 is indexed once; layers 1 and 3 are indexed per product and partitioned by product identifier. An agent identity belongs to one product; cross-product retrieval exists only for the architect's assistant agent and the organizational drift audit.

26 Operating several products with one team

AICD is designed for a small human team supervising several products, each potentially on a different stack. The structure that makes this workable:

- **One organizational repository** (layer 2): the methodology, the agent role definitions, the shared conventions for deployment, observability, secrets, tickets and pull requests, the security policy, the calibration procedures. Every product inherits it.
- **One fleet instance per product**, with its own agent identities, memory scopes, ticket queue and continuous test environment. Fleets never share credentials or context. A product's coder cannot see another product's code.
- **Product-specific stacks, shared operational shape**. A product may run on any stack (section 20), but every product exposes the same operational surface: the same deploy-from-tag procedure, the same rollback semantics, the same observability signals, the same ticket format, the same pull request report. The humans then supervise all products the same way regardless of language or framework.
- **Seats span products**. The three seats and the product owner role exist once for the organization; each seat covers all products, with rotation (section 19) to keep judgment broad. A product may have its own product owner.
- **One review queue, ordered by risk and by product priority**. Escalations and tier 1 and 2 approvals from all products land in one queue, ordered by risk tier first, then by the organization's product priority list, then by age. Incidents jump the queue.
- **Cross-product incidents**. When several products share a component or a third-party service, an incident on it is opened once, linked to every affected product, and handled by the reliability and governance seat with the operations agent of each product executing its own runbooks.
- **Portfolio dashboard**. Section 30 describes the per-product view; the organizational view aggregates queue depth, escalation rate, cost, and incident state across products, so that a supervisor can see in one screen which product needs attention.

27 Security and compliance in depth

Threat model of the agent fleet

Threat	Path	Controls
--------	------	----------

Injection through production data	A user triggers an error or submits content containing instructions; the QA, operations or product signal agent reads it as telemetry	Production data is presented to agents as quoted, typed data with provenance, never as free text in the instruction channel; agents that read production have no write path to code; every action on a production-derived ticket is reviewed by the lead agent before a coder sees it
Injection through dependencies and documentation	A library, a README or a web page read by a coder contains instructions	Coders fetch documentation only from an allow-list; new dependencies are an escalation trigger; dependency contents are data, not instructions
Compromised agent identity	An agent's credential leaks or its runtime is compromised	Short-lived credentials, per-agent identity, least privilege per the permission matrix; a compromised coder can at most open a branch; a compromised QA agent can at most write to staging; anomaly detection on the audit trail
Supply chain	A malicious or vulnerable package enters through a coder's change	Dependency audit on every pull request; new dependencies escalate; the dependency register (section 24) is kept current; pinned versions; license check
Insider	A human with a seat abuses it	Tier 2 requires two humans; the audit trail covers human actions too; permission changes require the reliability and governance seat and are themselves audited
Model provider	Provider outage, model behavior change, data exposure to the provider	Provider-agnostic agent layer; calibration re-run on any model change; data classification decides what may be sent to which provider; the test environment uses anonymized data only
Agent collusion or drift	Coders and reviewer converge on the same wrong belief	Cross-model review; human reads tier 2 diffs; monthly human architecture review; mutation testing

Secrets architecture

- A single secrets manager is the only place credentials live. Nothing in repositories, prompts, instruction files or memory.
- Agents receive credentials at runtime, scoped to the task and expiring with it. The permission manifest (section 23) defines which identity may request which secret.

- Every secret has an owner (a seat), a rotation period, and a last-rotated date visible on the dashboard.
- A credential that appears anywhere it should not (a log, a ticket, a pull request, a memory record) is rotated immediately and the leak is an incident.

Data handling

- The data classification of section 24.3 (personal, financial, credentials, content) governs three things: what may be copied to the test environment (only anonymized), what may be sent to a model provider, and which modules are tier 2.
- Anonymization is a runbook executed by the operations agent, verified by the QA agent, and re-run on every refresh of staging data.
- Retention rules are part of the data inventory and are enforced by scheduled jobs that are themselves specified and tested.

Audit and compliance

The audit trail (every agent and human action, with identity, ticket, tool, input, output and time) is immutable and retained according to the organization's policy. Combined with the traceability chain of section 13 (line to commit to ticket to specification to decision), it answers the questions any compliance framework asks: who changed what, why, who approved it, and what proved it was correct. Regulated products add their framework's specific controls as an extension in organizational knowledge; the core provides the evidence.

28 The fleet dashboard

Humans supervise through a dashboard, not through terminals. The dashboard is designed around the four human functions and deliberately hides what would tempt a human into micromanaging the agents.

What is shown

For the function	What is visible
Supervisor	Per product: queue depth by category, tickets in progress with time in state, blocked items, escalations waiting, pull requests awaiting approval by tier, cost this month against the calibrated expectation, deployment state and last rollback test. Portfolio view across products.
Controller	The review queue with, for each item, the agent's report, the coverage matrix, the CI results, the mutation score delta, the lead agent's recommendation, and a one-click path to the diff for tier 2. Test plans awaiting verification.

Co-architect	Escalations with their context package, the ADR touched, the lead agent's question and recommendation. The drift audit results. The inherited ADRs marked for revisit.
Product owner	Product signal tickets, feature tickets by state, what shipped this week in product language.
Reliability and governance	Incidents, service level objective state, agent identities and their last credential rotation, permission changes, audit trail search, forbidden-action test results, the significant-modification triggers fired and the QA runs they caused.

What is deliberately not shown

- Agent working memory and intermediate reasoning. Watching an agent think invites intervention; the escalation protocol is the intervention path.
- Raw production telemetry inside the review interface. Humans see the QA agent's evidence attached to tickets, in a quoted and labeled form, not live logs.
- Vanity metrics: number of lines produced, number of commits, number of tickets closed without their category and tier.

29 Evolving the methodology: the extension process

Section 5 states that AICD is open to extension and closed to modification. This section makes it a procedure.

What may change, and how

Category	Examples	Procedure
Parameters	Budget multiples, review cadence, coverage and mutation thresholds, autonomy step durations	Decided by the owning seat, recorded in organizational knowledge, no proposal required. Re-measured after any model change.
Tooling	Which models run which roles, observability platform, ticket system, memory service implementation	Recorded as an organizational ADR by the owning seat.
Extensions	A new agent role, a new ticket category, a new test family, a new document type, a compliance overlay	Requires an AICD Change Proposal (below), reviewed by all seats, adopted with a version increment.

Core	The founding principles, the five layers, separation of duties, the tier structure, the migration rule	Not changed by a team. A team that believes the core is wrong documents the evidence from a real project and proposes it to the methodology's author as a candidate for a new major version.
------	--	--

The AICD Change Proposal

A change proposal is a document in organizational knowledge with: the problem observed on a real project, with evidence from the metrics of section 21; the proposed extension; which founding principle it serves; a demonstration that it violates none; what compensation structure, if any, it replaces and the evidence that the limitation is gone; the parameters it introduces and how they are calibrated. It is adopted when every seat has reviewed it and the reliability and governance seat has confirmed it changes no permission without a corresponding control.

Versioning

The methodology carries a version. Parameter and tooling changes do not change it. An adopted extension increments the minor version. A core change, which only the author makes, increments the major version. Every product records which version of AICD it runs, and a product is migrated to a new minor version by adopting the extension; there is nothing else to do, because extensions never invalidate what a product already does.

30 Calibration procedures

Several thresholds in AICD depend on the models in use and on the product. Section 21 defined performance-relative budgets. This section defines the other calibrations. All of them share the same shape: a calibration set with known answers, a measurement, a threshold expressed relative to the measurement, and a re-run on every material change.

Ticket categorization

1. During migration phase M4, or during the first steady-state month of a new product, every ticket filed by an agent is categorized by the agent and, in parallel and blind, by the verification lead.
2. The disagreement rate is measured, separately for dangerous disagreements (the agent under-categorized) and safe ones (the agent over-categorized).
3. Automatic validation of Auto and Behavioral tickets is enabled only when the dangerous disagreement rate is below the threshold the verification lead sets. Until then, every category is confirmed by a human.

4. The upgrade-only rule makes safe disagreements harmless; the calibration targets only the dangerous ones.

Escalation triggers

1. Record every escalation with its trigger and its outcome: the human agreed with the lead agent's recommendation, disagreed, or judged the escalation unnecessary.
2. Record every tier 1 and 2 pull request in which a human found something the lead agent should have escalated.
3. Triggers that produce mostly unnecessary escalations are narrowed; classes of change that produced missed escalations get a new trigger. Both changes are parameter changes recorded in organizational knowledge.
4. The target is not zero escalations; it is that every escalation is a real decision and no real decision is missed.

Proposed criteria and simulation findings

Two rates from the continuous test environment are tracked from the first run: the share of QA-proposed criteria the verification lead accepts as written, rewrites, or rejects; and the share of simulation findings a human judged not to be defects. A high rejection rate on either narrows the persona set, the significance triggers or the QA agent's instructions; a production defect the simulation should have found widens them. Both are re-measured after any change to personas, triggers or the QA model.

Retrieval quality

A calibration set of context questions (for a representative sample of closed tickets: which specification sections, ADRs, past tickets and modules were actually needed) is run against the memory service whenever the indexer, the embeddings, the code map builder or the model changes. Precision and recall of the returned context package are measured; a drop is treated as a defect of the memory service and blocks the change.

Significant-modification triggers

The list of what counts as a significant modification (section 15) is calibrated backwards: every defect found in production is traced to the change that introduced it, and if that change did not trigger a QA run, the trigger list is extended. Every QA run that finds nothing new is recorded; a trigger that never finds anything is narrowed.

PART VIII

Extensions and profiles

The remaining explorations of the methodology, and the profiles that adapt it to situations its core does not assume: products with mobile clients, environments without live telemetry, regulated domains, and a single operator. Ends with what the first real application taught, and what is still open.

31 Simulation personas and finding quality

Deriving personas from real usage

A persona is a synthetic user whose behavior is modeled on a real segment, so that a simulation exercises the product the way real users do rather than the way a test author imagines. Personas are derived from product analytics in four steps, once per quarter or when segments visibly shift:

1. **Segment.** The QA agent queries analytics for behavioral clusters: first-session users, returning users, power users, users of a particular journey (payment, upgrade, export), device and network profiles, and drop-off cohorts. Segments are chosen for behavioral difference, not demographics.
2. **Characterize.** Each segment becomes a persona definition: goals, the journeys it performs and in what order, session frequency and length, device and network profile, tolerance for failure (does it

retry, abandon, or contact support), and the data it typically has (an empty account versus three years of history).

3. **Review.** The verification lead and the product owner review the definitions. A persona that no real segment corresponds to is removed; a real segment without a persona is added.
4. **Version.** Personas live in `spec/criteria/personas.md`, versioned like every specification, referenced by identifier from simulation findings.

How many runs

A QA run is sized by coverage, not by a fixed number: every persona through every journey it performs, with enough variation seeds (different data states, timings, network conditions) to exercise the paths that matter. The stop criterion is saturation: when a further batch of runs produces no new unique finding, the run ends. The run budget (section 15) caps the worst case. The first runs on a product are large and later ones are smaller, because saturation arrives faster on a system with fewer defects.

Separating findings from noise

Mechanism	What it does
Fingerprinting	Every finding is reduced to a signature: journey step, error class, normalized stack or response fingerprint. Findings with the same signature are one ticket with a count, not many tickets.
Reproducibility gate	A finding must reproduce on replay before it becomes a ticket. What does not reproduce goes to a flaky-behavior list, which is itself reviewed, because intermittent failures are sometimes the most important ones.
Severity by user impact	Severity is computed from what the persona could not complete and how many real users the corresponding segment represents, not from the agent's impression of how serious it looks.
Known-issue suppression	Signatures that match an open ticket increment it rather than creating a duplicate; signatures that match a closed ticket reopen it, which is a regression signal.
Proposed criteria stay proposed	Section 15: no finding adds a criterion to the matrix without the verification lead.

Calibration (section 30) tracks two rates: findings that a human judged not to be defects, and production defects the simulation should have found. Both feed the persona and trigger lists.

32 Agent instruction design

Every agent runs from an instruction file. These files are organizational knowledge, versioned in the organizational repository, and they are short: an agent obtains its knowledge of the product through the memory service, not by having the specification pasted into its instructions.

Three layers of instruction

- **Organizational base:** what every agent in the company is, the founding principles it operates under, the untrusted-input rule, the budget and blocked-report behavior, the output formats, and the absolute prohibitions. One file, owned by the architect seat.
- **Product base:** what is specific to this product and true for every role in it: where the specification lives, the tier 2 modules by name, the load-bearing paths an agent must never infer as dead, the stack, how to run the tests. One file per product.
- **Role file:** the role's mission, what it reads and may write, its escalation triggers, its model, and the parts of the base it overrides. One file per role per product, derived from a shared role template.

Rules for writing them

- Rules, not descriptions. "Never modify an existing test; escalate instead" beats "be careful with tests".
- Prohibitions are explicit and enumerated. An agent cannot respect a boundary it was never told about; the load-bearing paths of a repository are named, with the evidence, so no future agent infers them as dead.
- Escalation triggers are listed with an example each.
- Every rule carries the methodology section it comes from, and a citation checker verifies the references (section 39).
- Instruction files are tested: the forbidden-action test proves the prohibitions hold, and the calibration set (section 30) is re-run when a file changes materially.
- Changing an instruction file is a parameter or extension change under section 29, never a silent edit.

33 Design in the loop

Design artifacts are part of the specification and receive the same discipline: versioned, referenced, verified against the built product.

- **Versioned.** Every screen, flow and component design is exported as a file with a stable identifier into `spec/design/`, alongside a link to its editable source. The exported file is what the agents consume and what the drift audit compares against; the source is for humans.
- **Referenced.** Each PRD journey names the design identifiers it is built from; acceptance criteria that concern appearance or layout reference the same identifiers. A design without a journey, or a journey without a design, is a divergence.
- **Verified.** The QA agent compares the built product against the design: structural checks (the components the design specifies are present and in order), visual comparison with a tolerance, accessibility checks, and responsive behavior at the breakpoints the design defines. Failures are findings like any other.
- **Changed through tickets.** A design change that alters a journey is decisional; one that alters appearance only is behavioral; a design produced for a new roadmap phase is part of that phase's set (section 23). The product owner approves designs; the verification lead approves the criteria that reference them.

34 The reference organizational repository

Layer 2 of memory is a repository. This section fixes its shape so that every AICD organization's repository has the same structure and a product can be moved between them.

Path	Contents
<code>methodology/</code>	This document, its section index in machine-readable form, the version in force, and the adopted Change Proposals
<code>roles/</code>	The organizational base instruction file, the role template, and one reference role file per agent role
<code>templates/</code>	Ticket, pull request report, acceptance criterion, ADR, blocked report, post-mortem, escalation, Change Proposal, calibration record
<code>gates/</code>	Tool-agnostic definitions of each gate (what it checks, what input, what constitutes failure, the planted defect that proves it), with example implementations per stack

<code>policies/</code>	Security policy, stack selection policy, the significant-modification list, the adversarial review checklist, the data classification scheme
<code>calibration/</code>	Calibration set formats, procedures, and the organization's current measured baselines
<code>profiles/</code>	The extension profiles of this Part, and any the organization has adopted
<code>products.md</code>	The portfolio: every product, its AICD version, its stack ADR, its fleet, its seats

The first instance of this repository is produced by the first migration an organization runs, from the artifacts that migration creates. The methodology's next version will ship a minimal reference instance.

35 Profile: products with mobile clients

The core of AICD assumes that shipping is instantaneous and reversible: merge, tag, deploy, and if it fails, redeploy the previous tag. A native mobile client breaks every part of that assumption. Release goes through a store review measured in hours or days. Users update when they choose, so several versions run at once for weeks. There is no redeploying a previous tag: the only way to change what runs on a device is to ship a new version through the same slow path. And the client binary is in the hands of anyone who wants to read it. This profile adapts the methodology to those facts without weakening it.

What changes

Core assumption	Mobile reality	AICD adaptation
Deploy is instant	Store review, then staged rollout over days	The "ship" step becomes a release train (below). The tag is a release candidate; promotion goes through internal testing, beta, staged percentages, then full availability.
Rollback is redeploy the previous tag	Cannot force a device to downgrade	Rollback is redefined as: halt the staged rollout, and disable the behavior server-side. Every behavioral change in the client ships behind a remotely controlled flag with a kill switch. A change that cannot be disabled remotely is tier 2 by definition.

One version in production	N versions live at once	A compatibility window ADR: which client versions the backend supports, for how long, and the forced-update policy. Contract tests run both ways: new client against current backend, and every supported client version against the new backend.
Secrets on the server	The binary is readable	Nothing secret in the client. Vendor keys are held by the backend; the client authenticates to the backend only. Signing keys and store credentials are tier 2, human-held, used by CI through the secrets manager.
Verification runs on one platform	Many OS versions and devices	The continuous test environment includes a device matrix (emulators and real devices) chosen from the analytics device profile; on-device end-to-end tests are a required test type.
External constraints are your own	Store policies govern permissions, privacy labels, payments	Store policy is treated as an external contract. Any change to permissions, entitlements, data collection declarations, in-app purchase, push configuration or deep links is decisional.

The release train

Because each store submission costs days, client changes are batched into trains on a fixed cadence (weekly or biweekly), and the train is the unit that goes through the full QA run. Steps:

1. **Freeze.** The train's tickets are merged; the release branch is cut from the tag. Anything later waits for the next train, unless it is an incident.
2. **Candidate verification.** The exhaustive QA run executes against the candidate on the device matrix, plus the two-way compatibility tests, plus a store-compliance checklist maintained in organizational knowledge.
3. **Internal and beta tracks.** The candidate goes to the internal track, then to a beta cohort. Crash-free sessions and journey completion on the beta cohort are gate metrics.
4. **Submission.** The operations agent prepares store metadata, changelog and screenshots from the tickets; a human submits. A store rejection is handled as an incident with its own runbook, since it blocks every ticket in the train.
5. **Staged rollout.** Availability rises through percentages on a schedule. The operations agent holds the rollout automatically if crash-free sessions or a journey completion rate on the new version fall below the threshold, and flips kill switches for the behaviors that the failing version introduced.
6. **Full availability and observation.** The version is fully released; the adoption curve is tracked; old versions leave the compatibility window on the ADR's schedule.

Additional risk tiers and triggers

- Tier 2, in addition to the core list: any change not behind a remote flag; native modules and platform SDK upgrades; the minimum supported OS or client version; signing, entitlements, permissions, privacy declarations; in-app purchase and subscription logic; push notification setup; deep link handling; offline data storage and migration on device.
- Significant modification, in addition to the core list: any change to the build configuration, to a native dependency, to on-device storage, or to the compatibility window.
- Escalation trigger: any client change that requires a backend change to work, because it forces coordination across two release cadences.

Verification specifics

- On-device end-to-end tests are mandatory for every journey, on the device matrix.
- Upgrade tests: install the previous version with realistic on-device data, upgrade to the candidate, verify data and session survive. On-device migrations are treated like database migrations: backward compatible and rehearsed.
- Offline and degraded-network simulation is a required persona dimension.
- Binary inspection is a required security check: no secrets, no debug endpoints, no verbose logging in release builds.

Observability specifics

- Service level objectives are per version: crash-free sessions, journey completion, and API error rate by client version.
- Client telemetry is batched and delayed; the operations agent's incident logic accounts for the lag rather than reading silence as health.
- The kill switch and rollout halt are runbooks the operations agent may execute; re-enabling is human.

Web clients with the same problem

A web application with a service worker, an installed progressive web app, or long-lived client sessions has a milder version of every issue above: old client code keeps running after a deploy. The same remedies apply in proportion: versioned APIs with a compatibility window, flags for behavioral changes, and a forced-refresh mechanism.

36 Profile: operating without live telemetry

The core assumes the operations, QA and product signal agents read live production telemetry. Some deployments forbid it: on-premises installations, air-gapped networks, zero-telemetry privacy commitments. Layer 4 then has to work on what can be exported rather than on what can be observed.

- **Diagnostic bundles.** The product produces, on operator request or on a schedule, an encrypted bundle of structured diagnostics: error records already passed through the sanitization barrier (section 8), aggregate metrics, configuration fingerprint, version. The bundle schema is part of the specification. The operations agent ingests bundles asynchronously through a bridge that enforces the untrusted-input rule and the barrier a second time.
- **Local simulation mirror.** The continuous test environment is instantiated inside the disconnected perimeter, with its own QA agent running on models permitted there, producing findings as files that leave the perimeter as part of the bundle. Personas are derived from whatever aggregate usage the operator is willing to export, or from the operator's description when nothing can leave.
- **Feedback loop with lag.** The loop of section 16 still closes, on the cadence of bundle delivery. Incident detection becomes the operator's responsibility, supported by an on-site runbook; the methodology's contribution is that every fix still ends in a specification update and a new criterion.
- **Model choice.** The permitted model provider inside the perimeter, possibly a self-hosted model, is a stack ADR with the calibration procedure re-run for it.

37 Profile: regulated domains

Regulated products (payments, health, finance, public sector, personal data under strict regimes) do not need a different methodology; they need the methodology's evidence mapped to their framework's controls, plus a few additions.

Evidence mapping

Control family (any framework)	AICD evidence
Change management	The traceability chain: line to commit to ticket to specification section to human approval, with the tier and the reviewers, for every change
Separation of duties	Coder, reviewer, tester and operator are distinct identities with distinct permissions; tier 2 requires two independent reviews

Access control	The permission manifest, per-identity short-lived credentials, the audit trail of permission changes, the forbidden-action test results
Testing and validation	Acceptance criteria, coverage matrix, mutation scores, QA run reports, the planted-defect demonstration for every gate
Incident management	Incident tickets, post-mortems, and the criteria and runbook changes they produced
Data protection	Data classification, anonymization runbooks and their verification, the sanitization barrier, the record of what is sent to which model provider
Logging and monitoring	The immutable audit trail and the observability specification

Additions for regulated products

- Any change touching regulated data or a regulated process is tier 2, whatever else it is.
- Data classification decides what may be sent to an external model provider at all. Where regulated data cannot leave, the agents work on anonymized or synthetic data only, or on models hosted within the permitted boundary.
- Retention and residency rules for the audit trail and operational memory follow the framework, and are themselves specified and tested.
- Evidence export is a tool: a human can produce, for any time window, the complete chain for every change, in a form an auditor can read.
- Human accountability is explicit: every tier 2 approval names the accountable human, and the methodology's agents are never the accountable party.
- The overlay itself is adopted as an extension through a Change Proposal, so that the framework's specific controls are versioned with the methodology.

38 Profile: the single operator

Tier 2 requires two humans. A single operator cannot provide that, and the core does not bend. This profile records the only accepted substitute, so that a solo operator runs the methodology with a documented, expiring exception rather than a silent gap. All of the following are required together for every tier 2 change:

1. Independent review by a different model than the coder, with a hard veto rather than an advisory opinion, working through the adversarial checklist.
2. The operator's own full diff read, in a separate session from the one in which the plan was approved, at least several hours later.

3. Rollback rehearsed on staging for that specific change before merge, and for changes with data migrations, a snapshot taken immediately before the migration with a rehearsed restore.
4. Migrations backward compatible with the previous tag, behind a flag where the change is behavioral.
5. Limited rollout where the product supports it, with the operations agent authorized to halt.

The exception is recorded as an ADR with status accepted and an expiry condition: the day a second seat is filled. Until then, the operator merges every tier 1 and tier 2 change personally; no agent merges above tier 0.

39 Lessons from the first application

The first migration under AICD, on a live product, produced findings that the methodology now carries as rules. Each is recorded here with the failure that earned it.

What happened	Rule adopted
A ticket instructed the deletion of a directory described as a dead legacy backend. The agent's first gate, prove which component serves production, showed the directory owned the live database's migration history and built two of five production images.	Destructive tickets state a belief and its source, never a conclusion, and proof precedes action. Any ticket that deletes, drops, revokes or rewrites begins with a discovery stage that changes nothing and ends with the human's decision. The ticket template carries this.
A rollback point was named in the backlog as if it existed. It did not.	Every precondition a ticket names is verified to exist at approval time, not at execution time.
Two CI workflows sat intact behind a manual trigger and a build target had no recipe, while every document cited them as gates. A newly written citation checker returned failure on clean and dirty trees alike and looked like it was working.	"Present but reporting nothing" is a named defect class. A gate is installed only after it has been seen to fail on a planted defect (section 14), every artifact that cites a gate names the trigger that fires it, and a liveness check fails loudly when an expected gate has not run inside its window.
Four drafting agents produced eleven fabricated section references, one of which would have removed a required check.	References are checked mechanically. The methodology's section index is machine-readable, a checker validates every reference in specification, operational memory and instruction files, and the drafting-error rate is a calibration metric.

An inventory of the system, containing no secret values, concentrated every account identifier, project reference and secret path onto one page.	Aggregation is classification. Identifiers that are individually harmless are stored apart from the documents that cite them, with their own memory scope, and the data classification names aggregation as a category.
Credentials for vendors no longer integrated were still live, some only in backup files, where no rotation cycle would ever reach them.	Revocation is a migration step distinct from rotation. Phase M1 revokes every credential whose integration no longer exists before rotating those that do, and the env report covers backup and archive files.
The repository had an undocumented dual-runtime architecture in which a "retired" service owned the schema. Nothing in the tree said so.	Load-bearing paths are named in the product instruction base, with evidence. An agent cannot respect a dependency it was invited to infer as dead.
Two migration chains wrote to one database with no down-migration and no snapshot, and had already produced one incident repaired by hand.	A deploy path with no way back is tier 2 and blocks agent autonomy on migrations until a pre-migration snapshot with a rehearsed restore exists.

The pattern across these is worth stating once: none of the failures were the agents' inventions. Every one was a human belief written into an artifact as a fact, or a control that existed on paper. The methodology's gates caught them because the gates demanded proof rather than agreement.

40 Open questions and next explorations

Version 0.3 closes the exploration list of version 0.2. What remains is validation and one deliverable.

To validate on real products

- The migration procedure end to end, with the actual duration of each phase recorded, on the product where it is now running.
- The mobile profile on a product with a native client, in particular whether the release train cadence and the rollout-halt thresholds are right in practice.
- The single-operator profile: whether its five substitutes hold up over months, and what the second seat changes when it arrives.

- The sanitization barrier against real production data, including the false-rejection rate on legitimate error messages.
- The calibration procedures, with measured rates rather than provisional ones.

To deliver

- The reference organizational repository of section 34, extracted from the first migration's artifacts.
- The adversarial review checklist as a versioned policy, seeded from the first post-mortems.

APPENDICES

Templates, glossary, history

A Templates

A.1 Acceptance criterion

ID	Product code, feature code, sequence number. Example: <code>PAY-REFUND-004</code>
Type	functional, security, performance, resilience, simulation
Precondition	The state of the system and the actor before the action
Action	What the actor does, in observable terms
Expected result	What must be observable afterwards, including side effects and what must not happen
Risk tier	0, 1 or 2, according to the area touched
Notes	Edge cases, related criteria, known limitations

A.2 Ticket

Title	One line, imperative
Category	Auto, Behavioral, Decisional, Product signal

Proposed risk tier	0, 1 or 2
Specification anchor	Section implemented, violated or to be modified
Acceptance criteria	Identifiers covered by this ticket
Evidence (defects)	Reproduction steps, logs, traces, session replays, severity, affected users
Budget	Max attempts, max wall-clock time, max tokens
Filed by	Human or agent identity
Closing requirements	Specification update reference, new criteria reference

A.3 Pull request report

Ticket and anchor	Links
Plan	The plan written before implementation, and deviations from it
Tests	Added and modified tests, with the reason for each
Coverage matrix	Criterion identifier to test name
Not tested	What was deliberately not tested and why
Risk tier	Self-assessed, with justification
Rollback plan	Required for tier 2
Escalations	Any trigger encountered and how it was resolved

A.4 Architecture decision record

Title and identifier	Example: ADR-012: Event store for order history
Status	Proposed, accepted, superseded (by), deprecated
Context	The forces at play: requirements, constraints, what prompted the decision
Options considered	Each option with its trade-offs
Decision	What was chosen and why
Consequences	What becomes easier, what becomes harder, what the agents must now respect
Escalation scope	Which changes in this area must be escalated to a human

A.5 Blocked report

Ticket	Link
Budget consumed	Attempts, time, tokens
Approaches tried	Each approach and why it failed
Hypothesis	Why the agent believes the ticket is blocked
What it would try next	With the reasoning
What it needs from a human	A decision, information, a permission, a specification clarification

B Glossary

Term	Meaning in AICD
Acceptance criterion	A structured, plain-language statement of one observable behavior the system must have. Authored by humans, mapped to tests by agents.
ADR	Architecture decision record. The document that records a significant design decision and defines what must be escalated in its area.
Agent	An AI system given one role, its own instructions, its own context and its own permissions.
Blocked report	The document a coder agent produces when it exhausts its budget, instead of continuing.
Canonical knowledge	The versioned specification of a product. The source of truth.
Coverage matrix	The table mapping each acceptance criterion to the tests that prove it. Enforced by CI.
Decisional ticket	A ticket that requires human approval before any work starts.
Drift audit	The weekly comparison of canonical knowledge against the actual code.
Escalation	A question raised by an agent to a human, with a recommendation, when a decision belongs to a person.
Fleet mode	The default mode in which coder agents work validated tickets autonomously under the lead agent.

Lead agent	The agent that orders, assigns and reviews the coders' work and escalates to humans.
Memory service	The tool connection through which agents access the unified memory, with scopes enforced.
Mutation testing	Introducing deliberate faults into the code to verify that the tests detect them.
Operational memory	The append-only log of tickets, incidents, decisions and attempts.
Pairing mode	A human directly driving a coding agent on one ticket.
Product signal	A usage insight that goes to the product owner rather than to the coder queue.
Risk tier	The 0, 1, 2 classification that determines merge requirements.
Separation of duties	The rule that the agent producing work never judges it.
Specification anchor	The section of the specification a ticket implements, violates or modifies.
Significant modification	A merged change that alters a journey, a business rule, a data model, a contract, a permission, an integration or a non-functional characteristic; triggers an exhaustive QA run.
As-built specification	Documents describing a product exactly as it exists, flaws included, produced during migration before any intended change is decided.
Characterization criteria	Acceptance criteria that describe current observed behavior, used as the verification baseline of a migrated product.
Calibration set	A small set of tickets or questions with known correct outcomes, used to measure model or memory-service performance and set relative thresholds.
Divergence register	The list of every difference between intended and as-built behavior, each resolved by a human decision.
Inherited ADR	A decision already embedded in a migrated codebase, recorded with status "inherited" and revisit conditions.
AICD Change Proposal	The document through which an extension to the methodology is proposed, reviewed and adopted.
Sanitization barrier	Deterministic code that converts production-derived data into typed records before it may enter operational memory; raw evidence is stored apart and never indexed.
Foundation set / phase set	The specification documents required before any code, and those written just in time per roadmap phase.
Release train	For mobile clients, the batched, scheduled release candidate that goes through candidate verification, beta, submission and staged rollout.

Kill switch	A remotely controlled flag that disables a client behavior without a new release; the mobile profile's substitute for rollback.
Present but reporting nothing	The defect class of a gate that exists and passes without checking anything, or never runs; the reason every gate must be seen to fail before it is trusted.
Upgrade-only rule	Agents may make a ticket safer (higher category) but never riskier; only humans downgrade.

C Document history

Version	Date	Changes
0.1	September 2026	First consolidated draft. Founding principles, five-layer architecture, agent team and separation of duties, memory architecture, specification system, feature flow, ticket categories, fleet and pairing modes with escalation protocol, git and release workflow, verification cycle and test types, continuous test environment, observability loop, permission model, human roles, hiring, stack policy, economics, failure modes, open questions, templates and glossary.
0.2	September 2026	Section 15 revised: continuous test environment becomes event-driven, exhaustive runs on significant modifications. Section 21 revised: no absolute cost baseline, performance-relative thresholds and calibration sets. New Part VI: starting a new product (gates G0 to G7, inherited golden rules, specification set, tag promotion replacing the three-branch model) and migrating an existing product (phases M0 to M5, access attribution at migration time, the fourteen as-built documents, verification baseline, progressive autonomy, exit checklist, special cases). New Part VII: memory service implementation, multi-product operation, security and compliance in depth, fleet dashboard, extension process and versioning, calibration procedures. Open questions rewritten. Glossary extended.
0.3	September 2026	External critique reviewed and adopted where warranted: three-layer review with deterministic checks and an adversarial checklist (section 7); sanitization barrier between telemetry and operational memory (section 8); conflict handling at scale with declared scope and a lock table (section 12); proposed criteria and QA run budgets (section 15); foundation set and phase set replacing the all-upfront specification (section 23). New Part VIII: simulation personas and finding quality, agent instruction design, design in the loop, the reference organizational repository, profiles for mobile clients, disconnected operation, regulated domains and the single operator, lessons from the first application recorded as rules, open questions rewritten. New rule in section 14: a gate is installed only when it has been seen to fail on a planted defect.

AICD: Artificial Intelligence Centered Development, by Alim Sahane. This document is a living methodology guide and is expected to evolve as the open questions in section 23 are explored and as the methodology is applied to real products. The founding principles and the structural core are stable; parameters and tooling choices are extension points.



ABOUT THE AUTHOR

Ngatcha Sahane Rachid Alim

Software engineer, ML engineer, and author of the AICD Methodology.

GitHub	github.com/rachidsahane
LinkedIn	linkedin.com/in/alim-sahane
Email	rachidsahane007@gmail.com

AICD grew out of building and operating several production SaaS products with AI coding agents, and out of the question that follows: how to organize a team that maintains and grows software no human wrote. The methodology is applied on live products and revised from what those applications teach. Feedback, critiques and reports of applying AICD on other products are welcome through any of the channels above.

AICD © 2026 by Alim Sahane is licensed under CC BY 4.0. To view a copy of this license, visit <https://creativecommons.org/licenses/by/4.0/>

Photographs and the author's likeness are excluded from this license and remain all rights reserved. The name AICD is subject to the trademark policy in TRADEMARK.md.